IBM

IBM Aspera®

Aspera Orchestrator Workflow Authoring Guide

IBM Aspera

2026/11/04



BUILD • AUTOMATE • ORCHESTRATE

Contents

1 Introduction **3**
1.1 Terminology . . . 3

2 Quick start **4**

3 Lifecycle **5**

4 Workflow file **6**
4.1 File types . . . 6
4.2 Structure . . . 6
4.3 Plain format . . . 7
4.4 Rails format . . . 7
4.5 Workflow attributes . . . 7
4.6 Whiteboard attributes . . . 8
4.7 Action templates . . . 8
4.8 Referring to existing entities . . . 9

5 Graph XML **10**
5.1 Skeleton . . . 10
5.2 Terminal nodes . . . 10
5.3 Step . . . 11
5.4 Prerequisites and branches . . . 12
5.5 Synchronization rules (AND vs OR) . . . 13
5.6 Inputs . . . 13
5.7 Outputs . . . 14
5.8 Parameters . . . 14
5.9 Value types . . . 15
5.10 Notes . . . 15

6 Inputs of a step **16**
6.1 Placeholders . . . 16
6.2 Plugin specifications . . . 17

7 Execution **20**
7.1 Statuses . . . 20
7.2 Retries . . . 20
7.3 Processing code . . . 20
7.4 Triggers . . . 21

8 Secrets **22**
8.1 Where a secret is stored . . . 22
8.2 Where a secret is displayed . . . 23
8.3 Secret references . . . 23
8.4 Recommendations . . . 23

9 Main action plugins **25**
9.1 CustomRuby . . . 25
9.2 Filter . . . 26
9.3 LocalFileWatcher . . . 26

9.4 SimpleParameterLookup . . . 27
9.5 Faspex5InboxWatcher . . . 28
9.6 AsperaNodeApiTransferOperation . . . 29
9.7 WorkflowLauncher . . . 29
9.8 ArrayFanout and Funnelin . . . 30

10 Recipes **31**
10.1 Recipe: minimal workflow . . . 31
10.2 Recipe: runtime parameter and step chaining . . . 31
10.3 Recipe: branch on a status . . . 32
10.4 Recipe: wait for a file . . . 33
10.5 Recipe: sub-workflow with outputs . . . 34
10.6 Recipe: one work order per file . . . 35
10.7 Recipe: queue and resource pool . . . 38
10.8 Recipe: lookup table . . . 40
10.9 Recipe: operator decision . . . 41
10.9.1 Custom form . . . 44
10.10 Update an existing workflow . . . 45

11 Authoring checklist **46**

12 REST API **47**
12.1 import_workflow . . . 47
12.2 export_workflow . . . 48
12.3 publish_workflow . . . 48
12.4 find_constraints . . . 48
12.5 import_with_constraints . . . 48
12.6 initiate . . . 49
12.7 Other workflow endpoints . . . 49
12.8 ascli commands . . . 49

13 Known issues **51**

Chapter 1

Introduction

This guide explains how to write an Aspera Orchestrator workflow as a file, import it, publish it and run it, without the graphical designer and without exporting an existing workflow first.

It is written for people and for AI assistants. Each rule gives the exact element, attribute or value expected by Orchestrator.

The content is derived from Orchestrator 4.1.6. The examples in Recipes were imported, published and executed on Orchestrator 4.1.6.

Related documents:

- Action Template Reference: attributes, inputs and outputs of every action plugin (generated from Orchestrator).
- Plugin specifications: the same as data (YAML), with the allowed values of attributes and the template settings each input and output depends on. Its header describes its fields. Read one plugin with `yq`: `yq '.AmazonS3Operation' plugins.yml`.
- Plugin Manual: functional description of each action plugin.
- Plugin Development Guide: how to write an action plugin.

1.1 Terminology

Term	Meaning
Workflow	A named process definition, identified by an integer `id` and a `portable_id` (UUID).
Whiteboard	One revision of the design of a workflow: the graph, stored as XML in attribute `xml_export`.
Publish	Build the runnable workflow (steps, inputs, outputs, parameters, branches) from a whiteboard. Only a published revision runs.
Step	A node of the graph that executes an action.
Action plugin	The type of a step, for example `CustomRuby`. Also called step type or action type.
Action template	A configured instance of an action plugin: a database record holding the plugin settings (its attributes). Each step references one template by `Step_type` and `Action_id`.
Embedded template	A template that belongs to one workflow. Its name is `W<workflow id>__<step name>`.
Global template	A template shared by workflows, identified by its name.
Parameter	A named value of the workflow. A runtime parameter can be set when starting the workflow.
Input, Output	Named values consumed and produced by a step, as declared by the action plugin.
Work order	One execution of a published workflow.
Work step	One execution of a step in a work order.

Chapter 2

Quick start

The following file is a complete workflow with one step that returns a message. It uses the plain format, which is the simplest to write.

```
---
- !ruby/object:Workflow
  attributes:
    name: hello
    run_as: SYSTEM
- !ruby/object:Whiteboard
  attributes:
    error_free: true
    xml_export: |
      <?xml version="1.0" encoding="UTF-8"?>
      <Workflow id="0">
      <Start x="100" y="50" id="start"><Parameters></Parameters></Start>
      <Error x="900" y="350"
 ↪  id="error"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Error>
      <End x="900" y="50" id="end"><Synch_factor>0</Synch_factor><Prerequisites><Prerequisite
 ↪  id="step1" status="Complete"/></Prerequisites></End>
      <Fail x="900" y="200"
 ↪  id="fail"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Fail>
      <Steps>
      <Step x="400" y="50" id="step1"><Action_id>1</Action_id><Step_name>Say
 ↪  hello</Step_name><Step_type>CustomRuby</Step_type><Synch_factor>0</Synch_factor><Prerequisites><Prerequ
 ↪  id="start"
 ↪  status="true"/></Prerequisites><Parameters></Parameters><Inputs></Inputs><Outputs></Outputs></Step>
      </Steps>
      <Parameters></Parameters><Notes></Notes>
      </Workflow>
- CustomRuby__1: !ruby/object:CustomRuby
    attributes:
      name: W0__Say hello
      execute_code: "outputs['message'] = 'Hello'"
      typed_outputs: "{'message'=>'string'}"
- CustomRuby__1: []
- {}
- CustomRuby: 0.7.0
```

Import, publish and run it with the IBM Aspera CLI (`ascli`):

```
ascli orchestrator workflows import hello.yml          # displays the new workflow, including its id,
 ↪  for example 239
ascli orchestrator workflows publish 239
ascli orchestrator workflows start 239 @json:'{}' @json:'{"step":"Say hello","variable":"message"}'
```

The last command waits for the work order to finish and displays `Hello`. The same operations with the REST API are described in REST API.

Chapter 3

Lifecycle

1. Write a workflow file (`.yml`), or a package (`.wkf`) that also contains sub-workflows and remote nodes.
2. Import it: Orchestrator creates a new workflow with a new `id`. Each step's action template is created (or mapped to an existing global template). Whether the workflow is created in draft state or as a published revision depends on the server option `publish_api_imported_workflow` (default: published, `revision_id = 1`).
3. Publish it: Orchestrator builds the runnable workflow from the last whiteboard revision.
4. Start it: Orchestrator creates a work order. Runtime parameters are given at this point.
5. Read the work order status and outputs.

Rules:

- Import and publish do not validate the graph. Publish only requires the whiteboard attribute `error_free` to be set. An invalid graph is published and fails at run time, or never ends. Use the authoring checklist.
- An import through `import_workflow` always creates a new workflow. If the name is already used, Orchestrator appends a space and `(import_1)`, `(import_2)`, and so on. The `portable_id` of the file is ignored and a new one is generated. The import in the web UI and `import_with_constraints` keep it instead, unless a workflow of the server already has it.
- To add a revision to an existing workflow instead, use `import_with_constraints` with `add as revision`. See Update an existing workflow.
- The API has no operation to delete a workflow: delete it in the user interface.
- Web UI: opened from the list of workflows, the designer draws the published revision from the published steps and from the positions in its XML (`WorkflowsController#edit`). The Revisions tab draws the XML of the chosen revision (`WhiteboardsController#show`). If the designer shows an empty canvas, the graph is not lost, and importing again does not help: a `Synch_factor` other than `0` has no position (see Prerequisites and branches); the Revisions tab still shows the graph. Never click "Publish" on an empty canvas: this publishes an empty graph. A revision imported through the API is shown as created by *Unknown user*.

Chapter 4

Workflow file

4.1 File types

Extension	Content
`.yml`	One workflow, as described in this guide.
`.wkf`	A gzipped tar archive containing a manifest file `<name>.txt` and the files it lists. The manifest is a YAML list of file names: the first one is the main workflow (`.yml`), the others are its dependencies: sub-workflows (`Workflow_*.yml`) and remote nodes (`Remote_node*.node`). `export_workflow` produces this format when dependencies are exported.

The file name given to `import_workflow` must end with `yml` or `wkf`.

4.2 Structure

A workflow file is a YAML sequence of exactly six elements, in this order:

#	Element	Content
1	Workflow	Workflow attributes: name, description, ... See Workflow attributes.
2	Whiteboard	The graph (`xml_export`) and its state. See Whiteboard attributes.
3	Action templates	A map: key `<StepType>__<N>`, value the template object. See Action templates.
4	Dependency list	A map: same keys as element 3, value the list of external entities used by the template (written by `export_workflow`).
5	Packed dependencies	A map of dependencies included in a `.wkf` package. `{}` for a `.yml` file.
6	Plugin versions	A map: plugin name to version. Informational: not checked on import.

Rule for element 4 and 5: element 4 is used only for the dependencies packed in element 5. When element 5 is empty (a `.yml` file), element 4 is ignored, and template attributes are imported as written: see Referring to existing entities. In a hand-written file, use `{}` or empty lists in element 4.

The file can be written in two encodings. Both are accepted by the import.

4.3 Plain format

The plain format is the old (Orchestrator 4.0) format. It is the simplest to write, and the one used in this guide.

A file is in plain format when no line matches `concise_attributes:`. On import, Orchestrator converts it to the current format, line by line, and then loads it (`Compatibility::WorkflowYmlUpdater`). That converter is line-based, so the layout rules matter:

- An optional first line `---`.
- Each of the six elements starts with a dash and a space (`-` then space) at column 0.
  - Element 1 starts with `- !ruby/object:Workflow`.
  - Element 3 starts with `- <StepType>__<N>: !ruby/object:<StepType>` for the first template. Each next template starts with a line `<StepType>__<N>: !ruby/object:<StepType>` (any indentation). The separator must be exactly `: !ruby/object:`.
- A line `attributes:` is optional and ignored.
- The other lines of a section are its attributes, one `name: value` per line, with the same indentation within a section. Block scalars (`|`) are allowed, for example for `xml_export`, and for template attributes such as `execute_code` (Ruby code) or a private key in PEM format. In a double-quoted value, YAML escapes apply, for example `\r\n` between lines of `parameters_associations` of `SimpleParameterLookup`.
- The class of a template is the part of its key before `__`.
- An unknown attribute name makes the import fail.
- In element 1, attributes with a null value are ignored.
- In element 3, attributes with a *blank* value are ignored: `null`, empty string, and also `false`. For a boolean attribute, write `1` or `0`, not `true` or `false`: `0` is kept, `false` is dropped (the database default then applies).
- Elements 4, 5 and 6 are required, even when empty.

On the server, the uploaded file is rewritten in the current format.

4.4 Rails format

This is the format produced by `export_workflow`: YAML serialization of Rails objects. Each attribute is an item of `concise_attributes`:

```
- !ruby/object:Workflow
  concise_attributes:
  - !ruby/object:ActiveModel::Attribute::FromDatabase
    name: name
    value_before_type_cast: My workflow
  - !ruby/object:ActiveModel::Attribute::FromDatabase
    name: label
  new_record: false
  active_record_yaml_version: 2
```

An attribute without `value_before_type_cast` is null. In element 3, each template has the same form, under its key (`- CustomRuby__2: !ruby/object:CustomRuby`).

To read an exported workflow, look for the attribute names given in this guide. To write a workflow, prefer the plain format.

4.5 Workflow attributes

Columns of a workflow, and how `import_workflow` uses them:

Attribute	Type	On import
`name`	string	Used. Suffixed with a space and `(import_N)` if already used.
`description`	text	Used.
`run_as`	string	Used. One of `SYSTEM`, `CREATOR`, `OPERATOR`.

Attribute	Type	On import
label	string	Used.
tags	text	Used. Stored as |tag1|tag2|.
templates	text	Used.
max_running	integer	Used. Maximum number of running work orders of this workflow.
cleanup_after_days	integer	Used.
purge_after_days	integer	Used.
user_lock, locked_by	boolean, integer	Used.
created_by	integer	Replaced by the importing user.
portable_id	string	Replaced by a new UUID.
source_revision	string	Replaced by <name>|<revision>.
comments	text	Replaced by Imported from file <path>.
id, folder_id, revision_id, generated_from, created_at, updated_at		Ignored. The folder is given by the import request.

The minimum is name, and run_as is recommended.

4.6 Whiteboard attributes

Attribute	Type	Rule
xml_export	text	Required. The graph. See Graph XML.
error_free	boolean	Required to publish. Publish returns false if it is null. Set it to true: it is not computed on import.
revision_id	integer	Optional. Defaults to 1.
created_by, workflow_id, revisions_log, id, created_at, updated_at		Ignored or replaced.

4.7 Action templates

Element 3 maps each key <StepType>__<N> to an action template.

- <StepType> is the plugin class name, for example CustomRuby, and must be the Step_type of the steps using it.
- <N> is a number unique within the file for that type, and must be the Action_id of the steps using it. On import, the template gets a new database id and the graph is updated: Action_id of steps, and action_id of Map_to and Carry_thru of type output.
- The value holds the template attributes: the plugin settings. The attribute names of each plugin are listed in the Action Template Reference.
- Attribute name decides how the template is imported:
  - W<any number>__<step name>: embedded template. It is created for the new workflow and renamed W<new workflow id>__<step name>. Use this form, with W0__ and the step name.
  - Any other name: global template. If a template of the same class and name exists and has the same attributes, it is reused. If it differs, a new template named <name> (import_N) is created. If none exists, it is created.
- Plugins that are not enabled on the server are enabled automatically by import_workflow. If that fails, the import returns the list of missing plugins.

How a template defines the inputs of its step is explained in Inputs of a step.

4.8 Referring to existing entities

A template refers to an entity of the server (a workflow, a remote node, a journal book...) by the value of one of its attributes. In a `.yml` file, element 4 is ignored (see Structure): the import keeps the value written, and does not check that the entity exists. So write the reference in the attribute, as the entity is known on the target server, and leave element 4 empty.

Entity	Template attribute	Value
Workflow	`SubWorkflow`: `workflow_id`, `workflow_portable_id`	Id of the workflow (see below for the portable id).
Workflow	`WorkflowLauncher`: `workflow_id`	Id of the workflow. Or `workflow_id` blank, `use_portable_id` set to `1`, and `portable_id`: the portable id of the workflow, looked up at execution.
Remote node	`remote_node`, and other attributes with widget `remote_node` in the plugin specification	Name of the node.
Journal book	Step element `Journaling` of the graph	Name of the book. The import creates it if missing.
Journal book	`FileJournalLogEntry`: `journal_book`	Name of an existing book. It is not created: the entries are then written without a book.
Mailer	`EmailNotification`, `UserInput`: `mailer_name`	Name of a mailer configuration. Blank: the first active one.
Global action template	Element 3	A template with the same class, name and attributes: see Action templates.

Ids and portable ids of workflows are given by `ascli orchestrator workflows list`. No API lists the other entities: ask the administrator of the server.

`SubWorkflow`: each time its template is saved, import included, `workflow_portable_id` wins: if it is the portable id of a workflow of the server, `workflow_id` is set to the id of that workflow; otherwise, if `workflow_id` is the id of a workflow, `workflow_portable_id` is set to its portable id. `import_workflow` gives a new portable id to each workflow it creates (see Workflow attributes): a portable id read on another server does not match. Write `workflow_id`, and leave `workflow_portable_id` blank. The import in the web UI and import_with_constraints keep the portable id of the file, unless a workflow of the server already has it (`ImportExporter.import_new`): with them, a portable id written in the template matches the workflow imported from the same file.

Packages (`.wkf`): with dependencies, `export_workflow` lists in element 4 each entity a template uses, for example `{entity: Workflow, id_key: id, id_value: 4, dependent_field: workflow_id}`, and packs sub-workflows and remote nodes (element 5). On import, each packed sub-workflow is imported first, and the attribute named by `dependent_field` is set to the `id_key` of the new entity. With `import_workflow`:

- Remote nodes of the package are not imported: the attribute keeps the node name, and the node must exist on the server. `import_with_constraints` imports or maps them (see import_with_constraints).
- Field `link_to_subworkflows` set to `true`: a packed sub-workflow that was already imported (a workflow with the same name and a `source_revision` equal to `<name>|<revision_id>` of the packed file) is used instead of a new copy. The value is evaluated as Ruby code: only `true` links, and in a package with sub-workflows, a word other than `true` or `false` makes the import fail.

Chapter 5

Graph XML

The whiteboard attribute `xml_export` holds the graph as XML. Validate a workflow file before import with the IBM Aspera CLI:

```
ascli orchestrator workflows check_import my_workflow.yml
```

5.1 Skeleton

```
<?xml version="1.0" encoding="UTF-8"?>
<Workflow id="0">
<Start x="100" y="50" id="start"><Parameters></Parameters></Start>
<Error x="900" y="350"
 ↪  id="error"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Error>
<End x="900" y="50" id="end"><Synch_factor>0</Synch_factor><Prerequisites>...</Prerequisites></End>
<Fail x="900" y="200" id="fail"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Fail>
<Steps>
  <Step ...>...</Step>
</Steps>
<Parameters>...</Parameters>
<Notes></Notes>
</Workflow>
```

General rules:

- The root element name is free, `Workflow` by convention. Its attribute `id` is replaced on import: use `0`.
- The elements `Start`, `End`, `Fail` and `Error` are required, exactly once each.
- At least one `Step` is required.
- `Parameters` (top level) and `Notes` are optional.
- Attribute `id` of every node is a string, unique in the graph, used to link nodes. The designer uses UUIDs; any unique string works, for example `start` or `step1`.
- Attributes `x` and `y` are the position of the node in the designer. They do not change execution.
- Usual XML escaping applies (`&`, `<`, `"`). Values of parameters and of inputs mapped to a value are also URL-decoded: `%XX` and `%uXXXX` sequences are decoded. Write `%25` for a literal `%` followed by two hexadecimal digits.
- On import, the graph is parsed and stored written again: root element `opt`, one element per line with indentation, attribute values in double quotes, and the new template ids in `Action_id` and in `action_id` of `Map_to` and `Carry_thru`. This is observed with both formats of file, and with `import_with_constraints`: the export of an imported workflow shows this form, not the one of the file.

5.2 Terminal nodes

Element	Required children	Meaning
`Start`	none (`Parameters` optional)	Entry point. Its `Parameters` links declare the global parameters (see Parameters).
`End`	`Synch_factor`, `Prerequisites`	Reached: the work order is `Complete`.
`Fail`	`Synch_factor`, `Prerequisites`	Reached: the work order is `Failed`.
`Error`	`Synch_factor`, `Prerequisites`	Reached: the work order is `Error`.

`Fail` and `Error` are also reached implicitly: when a work step ends with status `Failed` (or `Error`) and no step is connected to it for that status, the work order goes to `Fail` (or `Error`). So only success paths need to be connected to `End`; failures do not need explicit links. The status details of the work order are then `No behavior defined for step status change to <status> - in <step name>`. The reason of the failure is in key `step_status_details` of output `<step name>:Step_information`: `ascli orchestrator workorders output <work order id>` shows it.

5.3 Step

Each `Step` element has attributes `id`, `x`, `y`, and these children:

Child	Required	Content
`Action_id`	yes	Number `<N>` of the template key `<Step_type>__<N>`.
`Step_name`	yes	Name of the step, unique in the workflow. Used to name outputs (`<Step_name>:<output>`) and in `Map_to` of type `output`.
`Step_type`	yes	Plugin class name, for example `CustomRuby`.
`Synch_factor`	yes	Number of prerequisites to meet. See Prerequisites and branches. When it is not `0`, attributes `x` and `y` (position of the join symbol) are needed by the designer.
`Prerequisites`	yes (to be valid)	Links from previous nodes.
`Inputs`	yes (may be empty)	Inputs of the step. See Inputs.
`Outputs`	yes (may be empty)	Output settings and carry-throughs. See Outputs.
`Parameters`	no	Links to step parameters. See Parameters.
`Version`	no	Plugin version, informational.
`Pre_processing`	no	Ruby code run before the action. See Processing code.
`Post_processing`	no	Ruby code run after the action.
`Timeout`	no	Execution timeout in seconds, `0` for none. Default: timeout of the plugin, else the engine setting `default_step_timeout` (60 by default).
`On_timeout`	no	Status on timeout: `Pass` (Complete), `Fail` (Failed, the default), `Error` (Error), `Raise` (Error, and the work order timeout action runs).

Child	Required	Content
`Error_attempts`	no	Maximum attempts when the step ends with `Error` and a branch with status `Error` leaves it.
`Error_retryFor`, `Error_retryForUnit`	no	Retry period: value multiplied by unit (seconds).
`Error_retryDelay`	no	Delay between attempts, in seconds.
`Failed_attempts`, `Failed_retryFor`, `Failed_retryForUnit`, `Failed_retryDelay`	no	Same, for status `Failed`.
`comments`	no	Description of the step.
`weight`	no	Integer, used for progress display.
`Journaling`	no	Journal entry settings (`Book`, `Event`, `Package`, `File`, `File_path`, `File_status`, `JournalStepMilestone`, `JournalProcessingTimeout`). A missing journal book is created on import.

Element names are case-sensitive and must be written exactly as above. The parser ignores unknown elements: a misspelled optional element has no effect.

5.4 Prerequisites and branches

A `Prerequisite` element links a previous node to the current one:

```
<Prerequisites><Prerequisite id="check" status="Complete"/></Prerequisites>
```

Attribute	Content
`id`	`id` of the previous node.
`status`	Status of the previous node that activates the link. `true` for a link from `Start`. Otherwise the final status of the previous step: `Complete`, `Failed` or `Error` (compared case-insensitively).
`router`	Line style of the link in the designer (no effect on execution), chosen with a right-click on the link: `draw2d.ManhattanConnectionRouter` (right angles, the default when the attribute is absent), `draw2d.BezierConnectionRouter` (curve), `draw2d.NullConnectionRouter` (straight line), `draw2d.FanConnectionRouter` (straight lines spread apart when several links join the same two nodes). The designer uses the value as a JavaScript class name: no other value. Use Manhattan for step prerequisites and Bezier for parameter links; use Bezier on a step prerequisite only where the graph reads better, for example when a Manhattan line would cross a step.

`Synch_factor` sets how many prerequisites must be met:

Value	Meaning
`0`	All prerequisites (AND).
`N` > 0	At least `N` prerequisites. `1` is OR: for example `End` after alternative branches.
negative	Never met: do not use. For a value of `-N`, the engine requires the number of prerequisites plus `N`, which is more than exist, although "all but `N`" was intended. For "all but `N`", set the positive value: number of prerequisites minus `N`. The designer refuses negative values.

When `Synch_factor` is not `0`, the designer draws a join symbol, at the position given by attributes `x` and `y` of `Synch_factor`. Write them, on steps and on `End`, `Fail` and `Error`: `<Synch_factor x="920" y="70">1</Synch_factor>`. The engine ignores them, but without them, the workflow opened from the list of workflows shows an empty canvas (see Known issues).

A link from `Start` always counts as met.

A step can have several outgoing links (several steps referencing it): steps whose prerequisites are met run in parallel.

To branch on a condition, make a step end with `Complete` or `Failed`, and connect different steps to each status. The `Filter` plugin does exactly that with a Ruby expression; a `CustomRuby` step can set `@status`. See Recipe: branch on a status.

5.5 Synchronization rules (AND vs OR)

Choosing the correct `Synch_factor` on a join node depends on the execution pattern of incoming branches:

- Parallel nominal execution (`Synch_factor = 0` / AND): When two or more nominal branches execute simultaneously in parallel (e.g., a fan-out where several tasks run at the same time), set `Synch_factor` to `0` on the convergence step. The join node waits until all parallel branches have completed before proceeding.
- Mutually exclusive / Alternative branches (`Synch_factor = 1` / OR): When branches originate from mutually exclusive outcomes (e.g., an `if/else` condition, a route decision between alternative lanes, or a nominal branch vs an error/failure branch), only one branch runs per execution. Setting `0` deadlocks the workflow because it waits forever for the branches that were not executed. In this case, set `Synch_factor` to `1` on the join step (or `End`/`Fail`) so that it proceeds as soon as any one of the incoming branches completes.

5.6 Inputs

```
<Inputs>
  <Input name="person"><Map_to type="parameter" name="who"/></Input>
  <Input name="text"><Map_to type="output" name="greeting" step_type="CustomRuby" action_id="1"
 ↪  step_name="Greet"/><Processing>input_value = input_value + '!'</Processing></Input>
</Inputs>
```

Attribute `name` of `Input` is the name of an input of the step, as declared by the action template (see Inputs of a step). An `Input` whose name is not declared by the template is silently ignored.

`Map_to` gives the source of the value:

`type`	Other attributes	Source
`value`	`name`: the literal value	A constant. For an input of type `array`, a value not starting with `[` is wrapped into a one-element array.
`parameter`	`name`: parameter name	A parameter (see Parameters).

type	Other attributes	Source
global	name: parameter name	Same as parameter.
output	name: output name, step_name, step_type, action_id	An output of a previous step. step_name, step_type and action_id must be the Step_name, Step_type and Action_id of that step.

Optional child Processing: Ruby code that transforms the value (see Processing code).

Every required input of the template must have an Input element.

5.7 Outputs

The outputs of a step are those declared by its action template, plus Step_information. They all exist, whether or not the Outputs element lists them. Each output is a variable named <Step_name>:<output name>.

The Outputs element only adds settings:

```
<Outputs>
  <Output name="file_name"><Processing>output_value = output_value.upcase</Processing></Output>
  <Carry_thru type="output" name="file" step_type="ArrayFanout" action_id="33" step_name="Fan out"/>
</Outputs>
```

Element	Content
Output	Attribute name. Optional children: Processing (Ruby code transforming the value), Global (name of a global variable that also receives the value).
Carry_thru	Makes a variable available as an input of this step without the plugin declaring it: at run time, the value is in @inputs under <name> and <step_name>/<name>. Attributes: type (output or parameter), name, and for output: step_name, step_type, action_id.

An input can be mapped to an output of any step of the graph, not only of a step linked to it: the recipes queue, one work order per file and operator decision map outputs of the step before the previous one, without Carry_thru. The designer has a narrower rule: the mapping list of a step offers only the outputs of the steps linked to it, the variables they carry through, and the global parameters (WhiteboardsController#input_mappings). There, Carry_thru is the way to offer an output to the step after the next one. In a file, map the output directly.

Output Step_information is a hash with keys step_id, step_name, step_status, step_status_details, attempts, processingTime, elapsedTime, completionTime.

5.8 Parameters

A parameter is defined once in the top-level Parameters element, and linked to Start (global parameter) or to a step (step parameter):

```
<Start x="100" y="50" id="start"><Parameters><Parameter id="p_who"/></Parameters></Start>
...
<Parameters>
  <Parameter x="100" y="200" id="p_who">
    <Name>who</Name><Value>world</Value><Value_type>string</Value_type>
    <Runtime_override>true</Runtime_override><Runtime_optional>true</Runtime_optional>
  </Parameter>
</Parameters>
```

Definition (Parameter with attributes id, x, y):

Child	Content
`Name`	Name, used in `Map_to` of type `parameter` and when starting the workflow.
`Value`	Value at design time, or default value of a runtime parameter. Use JSON for `array` and `hash`, for example `["/tmp/a", "/tmp/b"]`.
`Value_type`	One of the value types.
`Runtime_override`	`true`: the value can be given when starting the workflow. Default `false`.
`Runtime_optional`	`true`: a runtime value is optional, and `Value` is used when absent. Default `false`.
`Comments`	Description.

Link (`Parameter` with attribute `id`, and `router="draw2d.BezierConnectionRouter"` for the designer) in `Start/Parameters` or `Step/Parameters`.

Rules:

- A parameter that is not linked to `Start` or to a step does not exist after publish: inputs mapped to it are silently not created.
- Runtime parameters are the inputs of the workflow: `workflow_inputs_spec` lists them, and `initiate` receives them as `external_parameters`.
- A step parameter is a parameter linked to a step instead of (or in addition to) `Start`. Linking a parameter to a step does not change its value or scope: the engine sets all parameters when the work order starts (`Variable.generate_all`), and an input of any step maps to the parameter by name. The designer uses step parameter links to draw the parameter node near the step, and offers a step parameter only in the mappings of the steps it is linked to. In practice, link a parameter to `Start` and map the input directly.

5.9 Value types

Type	Content
`string`	Text.
`int`	Integer.
`float`	Floating point number.
`flag`	Boolean.
`date`	Date and time.
`array`	List (JSON array).
`hash`	Map (JSON object).
`object`	Any serializable value.
`pwd`	Password, hidden in the user interface, and encrypted in the published workflow and in work orders. The `Value` of a `pwd` parameter is in clear in the graph XML and in exports: give secrets at run time. See Secrets.
`attachment`	File attachment.

5.10 Notes

`Notes` holds designer notes: `<Note id="..." note_id="..." x="..." y="..." collapsed="..."><Text>...</Text></Note>`. They do not change execution.

Chapter 6

Inputs of a step

The inputs of a step are declared by its action template, in method `inputs_spec` of the plugin: a map of required inputs and a map of optional inputs, each from name to value type. The Action Template Reference lists them for each plugin.

Many plugins derive inputs from some of their template attributes (`ActionTools#default_inputs_spec`):

- A text attribute left blank in the template becomes an input, so its value can come from the graph at run time. The Action Template Reference gives its name in column Input if blank.
  - Input name: the value of the plugin constant `VAR_<ATTRIBUTE>` if it exists, else the attribute name with its first letter in upper case (for example `file_path` gives `File_path`).
  - The input is optional if the plugin defines a constant `DEFAULT_<ATTRIBUTE>`, else required.
  - An attribute name ending with `_<type>` (for example `delay_int`) gives an input of that type, named without the suffix.
- A text attribute that is set can contain placeholders `<%= variable %>`: each one creates an input.
- A non-text attribute left null becomes an input of the corresponding type.

So the same plugin can have different inputs depending on its template. For example, `LocalFileWatcher` with `directory` left blank has a required input `directory`.

Plugins that let the template declare its inputs and outputs explicitly (for example `CustomRuby` and `Filter`) take them as a type map in text: JSON (`{"person":"string"}`) or Ruby hash syntax (`{'person'=>'string'}`).

Before calling the action, Orchestrator adds these inputs: `OUTPUTS` (names and types of the outputs), `WORKORDER_ID` (the work step id), `ATTEMPT`, `RUNNING_AS`.

6.1 Placeholders

A text attribute marked `placeholders: true` in plugins.yml can contain ERB placeholders. Each variable they name becomes an input of the step, and the plugin replaces the placeholders with the input values at run time. For example, an `EmailNotification` template with `template` (the message body) set to `Hello <%= user_name %>, <%= file_name %> is ready.` gives required inputs `user_name` and `file_name`.

Placeholders in an attribute that is not marked create no input. This is the case of some attributes that give an input when left blank.

These forms create inputs:

Placeholder	Creates input	Replaced by
`<%= name %>`	`name`	The value of input `name`.

Placeholder	Creates input	Replaced by
`<%= function(a, b) %>`	`a`, `b`	The result of `function` called with the values of inputs `a` and `b`. Arguments are input names, not literal values. `function` is a method of the plugin (for example `proxy_tag` of `ItunesTransporter`), `xml` (a hash as XML elements: `{'v'=>[1, 2]}` gives `<v>1</v><v>2</v>`), or a method with its class (for example `File.basename`).
`var(name)` in an expression	`name`	`var(name)` returns the value of input `name`, for example `<%= var(file_count).to_i + 1 %>`.

Any other Ruby code in `<%= ... %>` runs, but creates no input: in `<%= file_name.upcase %>`, `file_name` is not an input of the step, unless another placeholder declares it. Write `<%= var(file_name).upcase %>`. A function call creates inputs only when the placeholder is the call alone and all its arguments are names (`Payload#parse_payload`): `<%= File.join(dir, 'sub') %>` (a literal), `<%= File.join(File.dirname(path), name) %>` (a nested call) and `<%= File.basename(path).upcase %>` create no input. Write `<%= File.join(var(dir), 'sub') %>`. At run time, a variable that is not an input is `nil`, and the code fails, for example with `no implicit conversion of nil into String`. `var` is only for expressions: `<%= var(file_name) %>` alone fails, write `<%= file_name %>`.

The input created by a placeholder:

- Has the name written in the placeholder: letters, digits and `_`.
- Is a `string`, unless the name ends with `_<type>`: `<%= count_int %>` gives input `count_int` (the suffix is kept) of type `int`.
- Is required, unless the plugin defines a constant `DEFAULT_<NAME>`.
- Is not created if the name is a method of the template that returns a value: `name` (the template name) or another attribute of the template that is set. That value is used.

Some plugins scan placeholders with their own code instead of `default_inputs_spec` (for example `Filter`): the input is a required `string`, unless the template declares it otherwise (`mandatory_inputs` and `optional_inputs` of `Filter`).

Placeholders are scanned only in the value set in the template: the ones in `code_default` (attribute left blank) create no input.

At run time:

- An input without a value is replaced by an empty string.
- An array is written in Ruby notation: `[1, 2]`.
- `<% ... %>` tags (code without output, for example `<% if ... %>`) run only when the value also contains an expression such as `<%= var(file_name).upcase %>`: do not rely on them.

`FileGenerator` evaluates its whole template (`file_body_template`) as ERB, `<% ... %>` tags included, with the inputs declared in `required_inputs` and `optional_inputs`. An input is read through the missing-method mechanism of the plugin: an input named like a method that exists (of Ruby, or a global method of Orchestrator such as `md5`) calls that method instead, and the step fails, for example with "wrong number of arguments (given 0, expected 1..4)" (tested on 4.1.6). The form of 4.1.6 mentions a `{{variable_name}}` syntax: it is not supported.

6.2 Plugin specifications

The file plugins.yml gives, for each plugin, its template attributes and the inputs and outputs of its steps, with the template settings they depend on. They were found by calling methods `inputs_spec` and `outputs_spec` of the plugin on templates

with one or two attributes changed. Read one plugin with `ascli orchestrator plugins spec <PluginClass>`, or with `yq '.<PluginClass>' plugins.yml`.

Each plugin is a key, its class name: the `Step_type` of a step, and the class of its template (key `<PluginClass>__<N>`).

```
AmazonS3Operation:
  attributes:
    operation: {type: string, label: File Operation, widget: select, values: [upload, delete, list,
↪   ...]}
    aws_key: {type: string, hint: Enter AWS access key ID}
  inputs:
    Aws_key: {type: string, required: true, if_blank: aws_key}
    Local_file_path: {type: string, required: true, if_blank: local_file_path, when: {operation:
↪   {in: [upload, download]}}}
  outputs:
    ETag: {type: string, when: {operation: {in: [upload, copy object]}}}
```

Attributes (besides `name` and `comments`, common to all templates):

Field	Content
`type`	Database column type. In plain format, write a boolean as `1` or `0`.
`label`	Label in the template editor, when it is not the attribute name (`aws_key`: `Aws key`).
`widget`	When not text or boolean: `select`, `multiline_text`, `password`, `number`, `remote_node` (name of a remote node), `type_map` (`{'name'=>'type'}`), `list`, `hash`, `file`, `action_template` (name of a template).
`values`	Allowed values.
`dynamic_values`	`true`: allowed values come from the server or an external system.
`run_time_value`	For a `remote_node`: value that makes the node an input of the step.
`default`	Database default.
`code_default`	Value used by the plugin when the attribute is blank.
`hint`	Placeholder text in the template editor.
`format`	Language of the value (code editor).
`placeholders`	`true`: placeholders in the value create inputs (`<%= variable %>`, `<%= function(variable, ...) %>`, `var(variable)` in an expression). Not set: placeholders in the value are not scanned, even if the attribute gives an input when blank.
`declares`	The value declares inputs or outputs of the step (`required_inputs`, `optional_inputs`, `outputs`), as a type map or a list.

Inputs and outputs (a name in angle brackets is the value of that template attribute):

Field	Content
`type`	Value type.
`required`	Inputs: `true`, `false`, or the condition for which the input is required (`{if_blank: ...}`, `{if_set: ...}`, `{when: ...}`).
`if_blank`	Exists only when this attribute is blank (for an input: it replaces the attribute), or when all attributes of the list are blank.
`if_set`	Exists only when this attribute (or all of the list) is not blank.
`when`	Exists only for some values of attributes: `{<attribute>: {in: [...]}}` (`null`: attribute blank) or `{<attribute>: {not_in: [...]}}` (set to another value).
`conditional`	`true`: other template settings also decide whether it exists, the conditions given are not complete.

`if_blank`, `if_set` and `when` must all match, unless `conditional` is `true`.

Chapter 7

Execution

7.1 Statuses

A work step ends with one of the following statuses, which are matched by the `status` of prerequisites:

Status	Meaning
`Complete`	Success.
`Failed`	Functional failure (for example a filter evaluated to false).
`Error`	Technical error (exception, timeout with `On_timeout` set to `Error`).

While running, a work step can also be `Inactive`, `In Progress`, `Paused`, `Canceling`, and so on. A work order is `Complete`, `Failed` or `Error` when it reaches `End`, `Fail` or `Error`, and `Canceled` when canceled.

7.2 Retries

When a step ends with `Error` (or `Failed`) and a branch with that status leaves it, the step can be retried before that branch is followed: up to `Error_attempts` (or `Failed_attempts`) attempts, within `Error_retryFor` x `Error_retryForUnit` seconds, waiting `Error_retryDelay` seconds between attempts.

Rules (`Prerequisite.retry_needed?`):

- `0` means no limit, for the attempts and for the period.
- Both limits apply: retries stop at the first one reached.
- With `Error_attempts` `1`, or with neither attempts nor period, the step is not retried.

7.3 Processing code

Ruby code can be attached to a step, an input or an output. It runs inside Orchestrator: it has full access to the server.

Location	Runs	Variables
`Input/Processing`	When the input value is read.	`input_name` (read-only), `input_value` (set it to change the value).
`Step/Pre_processing`	Before the action.	`@inputs`: map of all input values, modifiable. `@work_order_id`, `@state_id` (work step id).
`Step/Post_processing`	After the action.	`outputs`: map of output values, modifiable. `@status`, `@status_details`: set them to change the step result. `@work_order_id`, `@state_id`.

Location	Runs	Variables
`Output/Processing`	When the output value is stored.	`output_name` (read-only), `output_value` (set it to change the value).

If processing code raises an exception, a warning is logged and execution continues with the unmodified value, unless the server option `strict_prepost_processing` is set, in which case the step ends with `Error`.

7.4 Triggers

Some plugins wait for an event instead of executing at once, for example `LocalFileWatcher`. They do not hold a worker while waiting.

If the template enables continuous monitoring (for example attribute `allow_multiple` of `LocalFileWatcher`, or `keep_ongoing` of `Faspex5InboxWatcher`), the work order is cloned at the trigger step each time the step ends with a status other than `Error` (`WorkStep`, `WorkOrder.clone_at_step`):

- The work order that detected the event continues with the next steps, with the outputs of the trigger.
- The clone is a new work order that waits for the next event. Its `comments` are `Derived from WorkOrder <id>, forked at '<step name>' (<step id>)`, and its `derived_from` and `master_id` are the id of the first work order.

So the work order that waits is always the last one, and its id changes after each event. The work steps of the clones keep the group of the first work order (`WorkStep#group`): with the default history scope `by workorder group`, all the clones share the history of past triggers (see LocalFileWatcher). To stop the monitoring, cancel that work order: `ascli orchestrator workflows workorders <workflow id>` lists the work orders, the waiting one is `In Progress` on the trigger step. If a workflow monitor auto-starts the workflow (Monitors), turn the monitor off first: otherwise it starts a new work order at its next check. When the trigger step ends with `Error`, no clone is created, and the monitoring stops.

Chapter 8

Secrets

Orchestrator has no secret store for workflows. A secret (password, API key, token, passphrase) reaches a step from a template attribute, from an input mapped to a value or to a parameter, or from an entity of the server, such as a remote node. These ways differ in where the value is stored in clear, and in who can read it. With the plugin `SecretReference`, a workflow holds a reference to a secret of an external store instead of the secret, see Secret references.

8.1 Where a secret is stored

Source of the secret	Storage on the server	In an export (`export_workflow`, export in the web UI)
Template attribute, for example with widget `password` in plugins.yml	Clear, in the template.	Clear.
`Map_to` of type `value`	Clear, in the graph XML of each revision.	Clear.
`Value` of a parameter of type `pwd`	Clear, in the graph XML of each revision. Encrypted in the published workflow.	Clear.
Runtime parameter of type `pwd`, given when starting the workflow	Encrypted, in the variables of the work order.	Not exported.
Remote node, referred to by name (see Referring to existing entities)	Encrypted, in the remote node.	The name only. A `.wkf` package contains the node, with its password encrypted with the key of the exporting server.
Secret reference `secret://...`, in any of the above	The reference only. The secret is in its store: HashiCorp Vault, an encrypted file, or a remote node.	The reference only.

Encrypted values are prefixed with `~p:`: AES-256-GCM, with a key derived from the `secret_key_base` of the server (`OrchCipher`). The designer hides the value of a `pwd` parameter, but sends it to the browser: any user who can open the workflow in the designer, or export it, can read it.

Push to Git (Orchestrator 4.1.7 and later): option Protect Passwords on Push of the Git settings replaces, in the pushed file, the `Value` of `pwd` parameters and the `Map_to` values of inputs of type `pwd`. The replacement is the value encrypted with the export token of the Git settings (prefix `~gx:`), or an empty string when no token is set. An import, of any kind, on a server with the same export token decrypts them. Template attributes are pushed in clear, and `export_workflow` does not use this option.

8.2 Where a secret is displayed

The work step page of the web UI masks:

- Inputs and outputs of type `pwd`. The type of an input is the one declared by the plugin, not the type of the parameter mapped to it.
- Inputs, outputs and carry-throughs whose name contains `password` (any case), when the server option `mask_password_variables` is set (the default).

Other values are displayed in clear, even when they come from a `pwd` parameter. For example, inputs `Aws_secret` (Amazon plugins), `Files_client_secret`, `Api_token` and `Apikey` are of type `string`: check the type and the name of an input in plugins.yml.

The API does not mask: it returns the decrypted values to the users who can view the workflow.

Endpoint	`ascli` command	Values
`work_step_status`	`ascli orchestrator workstep status <WORK_STEP_ID>`	Inputs, outputs and carry-throughs of a work step.
`work_order_output`	`ascli orchestrator workorders output <WORK_ORDER_ID>`	Outputs of a work order.
`workflow_inputs_spec`	`ascli orchestrator workflows inputs <WORKFLOW_ID>`	`Value` of the optional runtime parameters.

Logs:

- The request log of Orchestrator writes `[FILTERED]` instead of the value of a request parameter whose name contains `passw`, `secret`, `token`, `_key`, `crypt`, `salt`, `certificate`, `login`, `email`, `otp` or `ssn` (`config/initializers/filter_parameter_logging.rb`). This applies to the runtime parameters in `external_parameters`.
- The access log of the web server (nginx) records the URL, with its query string.

8.3 Secret references

The plugin `SecretReference` (repository `aspera-orchestrator-secret-reference`, for Orchestrator 4.1.6) lets a workflow keep a reference instead of a secret:

```
secret://<STORE>/<PATH>[?<OPTIONS>][#<FIELD>]
```

- A reference is the whole value of a Password parameter, a step input or a template attribute. In a text, a placeholder such as `<%= Db_password %>` adds the input `Db_password` to the step: with a reference as its value, the placeholder is replaced by the secret.
- When a step runs, the plugin reads the secret from its store: the step gets the value, and variables, outputs and saved states keep the reference. A secret rotated in the store is used by the next run of a step.
- Stores are named in a configuration file on each server. Types: `vault` (HashiCorp Vault KV v2), `file` (encrypted file made by `ascli`), `node` (`login`, `password` or `certificate_passphrase` of a remote node).
- The step Secret reference checks that a secret can be read, and gives its reference (output `Reference`) and its version, never its value.

Installation, configuration and the dashboard are described in the README of the repository.

8.4 Recommendations

1. Do not write a secret in a workflow file, in a template attribute, or in a `Map_to` of type `value`. Leave the attribute blank: it becomes an input of the step (see Inputs of a step), and map this input to a parameter.
2. Give the secret when starting the workflow: a runtime parameter of type `pwd`, required, without `Value`. The secret is then in no revision, export or Git push, and `workflow_inputs_spec` does not return it.
3. For the credentials of a server defined as a remote node, refer to the remote node by name. For email, refer to a mail server configuration (`mailer_name`).

4. Name a runtime parameter that carries a secret so that the request log filters it: with `password`, `secret` or `token` in its name, for example `db_password`.
5. In `CustomRuby`, declare the inputs and outputs that carry a secret with type `pwd`: they are masked on the work step page, and a `pwd` output is stored encrypted. A `pwd` input accepts a `string` value.
6. Pass the secret in the JSON body of `POST api/initiate`, never in the URL. `ascli orchestrator workflows start` sends the parameters in the body. Do not type the secret on the command line (shell history, process list): read the parameters from a file, or from an environment variable, with `@json:@file:<PARAMS_FILE>` or `@json:@env:<VARIABLE>`.
7. Do not put a secret in an output, in `@status_details`, in an exception message or in a log (`Rails.logger`, `puts`). Outputs are returned by the API, and the status details of a step are in its output `Step_information`.
8. Give the permission to view the workflow only to the users who may read its secrets: the API returns them decrypted.
9. With Git, enable Protect Passwords on Push and set an export token (Orchestrator 4.1.7 and later). It protects `pwd` values, not template attributes.

Example: `DatabaseQuery`, with attribute `database_password` left blank, has a required input `DB_password` of type `pwd`. Map it to a required runtime parameter:

```
<Start x="100" y="50" id="start"><Parameters><Parameter id="p_db_password"/></Parameters></Start>
...
<Inputs><Input name="DB_password"><Map_to type="parameter" name="db_password"/></Input></Inputs>
...
<Parameters>
  <Parameter x="100" y="200" id="p_db_password">
    <Name>db_password</Name><Value></Value><Value_type>pwd</Value_type>
    <Runtime_override>true</Runtime_override><Runtime_optional>false</Runtime_optional>
  </Parameter>
</Parameters>
```

Start it with the parameters read from a file that only its owner can read (`chmod 600`), containing `{"db_password":"<PASSWORD>"}`:

```
ascli orchestrator workflows start <WORKFLOW_ID> @json:@file:<PARAMS_FILE>
```

Chapter 9

Main action plugins

The Action Template Reference lists the attributes, inputs and outputs of all plugins. This section details the plugins most useful to write workflows.

9.1 CustomRuby

Runs Ruby code. Inputs and outputs are declared in the template.

Attribute	Content
`execute_code`	Ruby code of the action.
`mandatory_inputs`	Required inputs: type map, for example `{'person'=>'string'}`.
`optional_inputs`	Optional inputs: type map.
`typed_outputs`	Outputs: type map, for example `{'greeting'=>'string'}`.
`inputs_spec_code`	Instead of `mandatory_inputs` and `optional_inputs`: Ruby code returning a map (required inputs) or an array of two maps (required, optional).
`outputs_spec_code`	Instead of `typed_outputs`: Ruby code returning the output type map.
`validate_inputs_code`	Ruby code returning `true` if the inputs are valid.
`use_code_from_github`, `github_*`, `source_control_type`, `use_cached`, `verify_ssl`	Get the code from a GitHub or Bitbucket repository instead of `execute_code`.

Contract of `execute_code`:

- Input values are in the map `@inputs` (also `inputs`), for example `@inputs['person']`. An input can also be read as a method with its name, unless that name is also a method of the template: `name` returns the template name, not an input named `name`. Prefer `@inputs[...]`.
- Set outputs in the map `outputs` (or `@outputs`): `outputs['greeting'] = 'Hello'`. Only outputs declared in `typed_outputs` are stored.
- Optionally set `@status` to `Complete` (default), `Failed` or `Error`, and `@status_details` to a message. Any other status becomes `Error`. The message is also kept with `Complete`: it is the `step_status_details` of output `Step_information`.
- `return <status>, <message>, @outputs` ends the code at once, for example `return STATUS_FAILED, "not a .tar file", @outputs` (constants `STATUS_COMPLETE`, `STATUS_FAILED`, `STATUS_ERROR`). The code runs with `eval` inside the method of the plugin, so `return` leaves that method: the status is not checked, and without the third value the outputs are not stored.
- An exception ends the step with `Error`.
- Write to the Orchestrator log with `Rails.logger.info`. No method `debug` is defined: calling it raises an exception.

- `JSON` is loaded: `JSON.parse` and `JSON.generate` work without `require`.

Available in `execute_code` and in processing code (`Pre_processing`, `Post_processing`, `Input/Processing`, `Output/Processing`):

Available	Notes
Full Ruby stdlib	`require 'net/http'`, `require 'fileutils'`, etc.
`File.nakedname(path)`	Orchestrator extension: basename without the last extension. `File.nakedname("/tmp/file.avi")` → `"file"`. `File.nakedname("/tmp/file.avi.old")` → `"file.avi"`.
`File.nakeddir(path)`	Orchestrator extension: immediate directory name. `File.nakeddir("/tmp/big/file.avi")` → `"big"`.
`File.extension(path)`	Orchestrator extension: last extension without dot. `File.extension("/tmp/file.avi")` → `"avi"`.
`XmlSimple.xml_in(xml)`	Requires `require 'xmlsimple'` at the top of the code block. The gem `xml-simple` is bundled with Orchestrator.
All gems in the Orchestrator `Gemfile`	`rest-client`, `nokogiri`, `net-ldap`, `rubyzip`, `uuid`, `redis`, etc. Use `require` as usual.

9.2 Filter

Evaluates Ruby code. The step ends with `Complete` if the result is true (any value other than `false` and `nil`), `Failed` otherwise, and `Error` if the code raises an exception. Output `FilterResult` (`flag`) holds the result.

Attribute	Content
`command`	Ruby code: an expression, or several lines, whose last value, or the value given to `return`, is the result. `@status_details` set in the code replaces the message of the step.
`mandatory_inputs`, `optional_inputs`	Additional inputs: type maps.
`filter_outputs`	Additional outputs: type map.

The code reads its inputs in two ways:

- `inputs['name']`, for an input declared in `mandatory_inputs` or `optional_inputs`: the value keeps its type, and needs no quoting. For example `!inputs['asset_list'].blank?`.
- Placeholders: each creates a required `string` input, unless declared. They are replaced by text before the code runs: quote a string value, as in `'<%= Status %>' == 'ok'`. A value that contains a quote breaks the code.

9.3 LocalFileWatcher

Waits for files matching a pattern in a directory of the Orchestrator host. Output `FileName` is the full path of the file (`string`), or the list of paths (`array`) if `return_all` is set; `FileContent` holds the file content if `read_file` is set.

Attribute	Content
`directory`	Directory to watch. Blank: becomes input `directory`.
`file_name`	File name pattern (glob, or regular expression if `use_regex_matching`). Blank: becomes input `File_name`.
`allow_multiple`	Continuous monitoring: keep watching after a detection (see Triggers).
`remove_file`	`1`: delete the file once detected (move it, with `archive_directory`). `0`: keep it, and remember it (see below).
`archive_directory`, `archive_with_timestamp`, `keep_timestamps`	Archive directory: the file is moved there with `remove_file` `1`, copied there otherwise. Output `FileName` is then the path in the archive.
`cool_off`	Seconds a file must stay unchanged to be considered stable.
`polling_frequency`	Seconds between scans (default 5).
`partial_file`	Trigger on files still being written.
`return_all`	Return all matching files at once.
`trigger_type`	Scope of the history of past triggers: `by workorder group` (default, also when blank), `by workflow`, `by action template`, `by step`, `absolute`, `none`.
`exclusions`, `fullpath_exclusions`, `ignore_zero_bytes`, `ignore_readlocked`	Filters.

History of past triggers: the watcher remembers the files it has detected, with their modification time, only when `remove_file` is `0` and `trigger_type` is not `none` (`LocalFileWatcher#some_persistence?`).

- `remove_file` `0`: a file is detected once per scope of `trigger_type`, and again when its modification time changes. With the default scope, the clones of a continuous monitoring share the history, and a new start (a new work order) detects again the files left in the directory.
- `remove_file` `1`: the file leaves the directory, no history is needed.
- `remove_file` blank (the database default; in plain format, also when written `false`): no history. A file left in the directory is detected again at each scan, by each new clone: with `allow_multiple`, a file that the processing does not remove starts a new processing every polling period.

Write `remove_file` explicitly: `1` to remove the file, `0` to keep it.

9.4 SimpleParameterLookup

Looks up a row in a table set in the template, and returns its values.

Attribute	Content
`parameters_associations`	The table: one row per line (LF or CR LF), columns separated by `:::`. The first line is the header: the names of the columns.
`param_file_name`	Instead of `parameters_associations`: path of a file on the Orchestrator host, with the same format, or CSV if its name ends with `.csv`.
`case_sensitive`	Compare values with case (default: ignore case).
`strip_spaces`	Remove spaces around values.
`disable_glob`, `disable_regex`	Do not match cells as patterns.
`return_array_of_rows`	Return all matching rows in output `rows`, instead of the first one.

Inputs and outputs come from the header:

- Each column name is an optional `string` input, and a `string` output.
- The inputs that have a value select the row: the first row whose cells, in those columns, match the input values. A cell matches if it is equal to the value, if it is a glob pattern that matches it (`File.fnmatch`), or if it is written `/<regular expression>/` and matches it.
- The outputs are the cells of that row, for all columns.
- If no row matches, the step ends with `Failed`: `No configuration match for data set: <column>: <value>`. With no input value (no key), no row matches.
- With `return_array_of_rows`, the step completes even if no row matches: output `rows` is then empty (and stays empty without a key).
- Input `ColumnToListValueFrom` (a column name) gives all the values of that column in output `ColumnValues` when the step completes: to list a column without a key, set `return_array_of_rows`.

Example: the step has input `Profile`, and outputs `FFMPEG Options` and `Target Extension` (plain format, rows separated by `\r\n` in a double-quoted value):

```
  SimpleParameterLookup__5: !ruby/object:SimpleParameterLookup
    attributes:
      name: W0__Transcoding profile
      parameters_associations: "Profile:::FFMPEG Options:::Target Extension\r\nMP4:::-vf
↪   scale=1280:720:::mp4\r\nXDCAM HD422:::-target pal-dvd -aspect 16:9:::avi"
```

9.5 Faspex5InboxWatcher

Waits for packages in a Faspex 5 mailbox. Each event is one package: the oldest one not seen yet.

Attribute	Content
`server`	URL of Faspex, for example `https://faspex.example.com/aspera/faspex`.
`user`, `client_id`, `private_key`	JWT authentication: the user whose mailbox is watched, the id of a Faspex API client with JWT enabled, and the RSA private key (PEM) whose public key is registered for that client. No client secret is used.
`mailbox`	`all` ("Global All", used when blank), `inbox` ("My Packages": packages received by the user), `inbox_all` ("All packages"), `shared_inbox` (with `shared_inbox_name`).
`keep_ongoing`	Continuous monitoring (see Triggers).
`new_only`	Ignore the packages already in the mailbox when the step starts.
`early_trigger`	`1`: trigger before the upload ends. `0`: trigger when the upload is complete and the package is released.
`polling_frequency`	Seconds between checks.
`get_transfer_spec`, `transfer_type`, `destination_root`	Output `Transfer_spec`: the transfer spec to download the package, for `connect` or `transfer_client`, with `destination_root` replaced by the attribute when it is set.

Outputs:

- `Id`: the package id, a number (as a `string`). `Package UUID`: its UUID.
- `File_list`: names of the files of the package, without path.
- `Transfer_spec`: JSON text from Faspex, with `direction` `receive`, `remote_host`, `remote_user`, `authentication` `token` and `token`, and one path: `paths[0].source` is the folder of the package on the Faspex storage, `<storage>/<title> - <UUID>.aspera-package/PKG - <title>/`. The token is stored in clear in the outputs of the work order.

To download the package to a node, give `Transfer_spec` as payload to `AsperaNodeApiTransferOperation`.

9.6 AsperaNodeApiTransferOperation

Starts a transfer on a node with the Node API, then checks its status every `polling_frequency` seconds (default 5) until it ends.

- With operation `Submit Transfer Using Payload`, input `Payload` (attribute `payload` blank) is posted as is to `/ops/transfers` of the node: a transfer spec as JSON text.
- The node is `https://<service_node_ip>:<service_node_port>` with `use_ssl` `1` (default port 9092), `http://` and port 9091 with `use_ssl` `0`, or the address of `remote_node`. `service_node_ip` can include the port (`<host>:<port>`). The certificate of the node is not verified.
- Authentication is Basic, with `user_login` and `user_password`: a node user and its password, or an access key and its secret. They are required also with `remote_node`. With wrong credentials, the step ends with `Failed`: `Transfer failed: 401 Unauthorized`.
- `destination_root` of the transfer spec is a path in the docroot of the node user, or in the storage root of the access key.

9.7 WorkflowLauncher

Starts a workflow in a new, independent work order.

Attribute	Content
`workflow_id`	Id of the workflow to start (see Referring to existing entities). Each runtime parameter of that workflow is an input of the step, with the same name, required unless the parameter is optional. Blank: input `Workflow_ID` (or the portable id, with `use_portable_id`), and input `Workflow_Parameters`, a hash of runtime parameters.
`wait_for_completion`	`1`: the step waits for the end of the launched work order. It polls: no worker is held.
`fail_if_failure`	`1`: with `wait_for_completion`, the step ends with `Failed` when the launched work order is `Failed`.
`work_order_name`, `master_id`	Label of the launched work order, and its master id, which groups work orders in the work order view. Blank: optional inputs `WorkOrder_Description` and `Master_id`.

Outputs: `WorkOrder_ID` (`int`), the id of the launched work order, and, when waiting, `WorkOrder_Status` (`string`), its status.

Status of the step (`WorkflowLauncher#execute` and `#check_status`):

`wait_for_completion`	Launched work order	Step
blank or `0`	Started	`Complete`
any	Could not be started	`Error`
`1`	`Complete`	`Complete`
`1`	`Failed`	`Failed` with `fail_if_failure` `1`, else `Complete`
`1`	`Error`, `Canceled`, rolled back, deleted	`Failed`, whatever `fail_if_failure`

9.8 ArrayFanout and Funnelin

`ArrayFanout` runs the next steps once per element of an array; `Funnelin` waits for all those executions and gathers their results.

- `ArrayFanout`: attribute `fanout_variable` is the name of the input (`array`), `fanned_as` the name of the output holding the current element, `variable_type` its type, `serialize` to run sequentially. Output `Fanout_Size` is the number of elements.
- `Funnelin`: attribute `funnel_variable` (and `funnel_type`) is the name of the input collected from each execution, `funneled_inputs` additional inputs (type map), `return_status` to add status outputs, `fail_if_failures` to fail if an execution failed. Output `Hash_by_state_id` maps each work step id to its value; each additional input `x` gives an output `x_hash`.

Exactly one step can be between them, and `Funnelin` must be linked to each end status of that step: see Recipe: one work order per file.

Chapter 10

Recipes

The following workflows were imported, published and run on Orchestrator 4.1.6.

10.1 Recipe: minimal workflow

See Quick start.

10.2 Recipe: runtime parameter and step chaining

Step `Greet` receives the runtime parameter `who`; step `Shout` receives the output of `Greet`, with input processing.

```
---
- !ruby/object:Workflow
  attributes:
    name: greet
    run_as: SYSTEM
- !ruby/object:Whiteboard
  attributes:
    error_free: true
    xml_export: |
      <?xml version="1.0" encoding="UTF-8"?>
      <Workflow id="0">
      <Start x="100" y="50" id="start"><Parameters><Parameter id="p_who"/></Parameters></Start>
      <Error x="900" y="350"
↪   id="error"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Error>
      <End x="900" y="50" id="end"><Synch_factor>0</Synch_factor><Prerequisites><Prerequisite
↪   id="shout" status="Complete"/></Prerequisites></End>
      <Fail x="900" y="200"
↪   id="fail"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Fail>
      <Steps>
      <Step x="300" y="50"
↪   id="greet"><Action_id>1</Action_id><Step_name>Greet</Step_name><Step_type>CustomRuby</Step_type><Synch_f
        <Prerequisites><Prerequisite id="start" status="true"/></Prerequisites>
        <Parameters></Parameters>
        <Inputs><Input name="person"><Map_to type="parameter" name="who"/></Input></Inputs>
        <Outputs></Outputs></Step>
      <Step x="600" y="50"
↪   id="shout"><Action_id>2</Action_id><Step_name>Shout</Step_name><Step_type>CustomRuby</Step_type><Synch_f
        <Prerequisites><Prerequisite id="greet" status="Complete"/></Prerequisites>
        <Parameters></Parameters>
        <Inputs><Input name="text"><Map_to type="output" name="greeting" step_type="CustomRuby"
↪   action_id="1" step_name="Greet"/><Processing>input_value = input_value +
↪   '!'</Processing></Input></Inputs>
        <Outputs></Outputs></Step>
      </Steps>
      <Parameters><Parameter x="100" y="200"
↪   id="p_who"><Name>who</Name><Value>world</Value><Value_type>string</Value_type><Runtime_override>true</Ru
```

```
      <Notes></Notes>
      </Workflow>
- CustomRuby__1: !ruby/object:CustomRuby
    attributes:
      name: W0__Greet
      mandatory_inputs: "{'person'=>'string'}"
      execute_code: "outputs['greeting'] = \"Hello #{@inputs['person']}\""
      typed_outputs: "{'greeting'=>'string'}"
  CustomRuby__2: !ruby/object:CustomRuby
    attributes:
      name: W0__Shout
      mandatory_inputs: "{'text'=>'string'}"
      execute_code: "outputs['result'] = @inputs['text'].upcase"
      typed_outputs: "{'result'=>'string'}"
- CustomRuby__1: []
  CustomRuby__2: []
- {}
- CustomRuby: 0.7.0
```

```
ascli orchestrator workflows start <id> @json:'{"who":"Laurent"}'
 ↪  @json:'{"step":"Shout","variable":"result"}'   # HELLO LAURENT!
ascli orchestrator workflows start <id> @json:'{}' @json:'{"step":"Shout","variable":"result"}'
 ↪  # HELLO WORLD!
```

10.3 Recipe: branch on a status

Step `Check` ends with `Failed` when the runtime parameter `value` is empty; one step is connected to each status; `End` waits for any of them (`Synch_factor` 1).

```
---
- !ruby/object:Workflow
  attributes:
    name: branch
    run_as: SYSTEM
- !ruby/object:Whiteboard
  attributes:
    error_free: true
    xml_export: |
      <?xml version="1.0" encoding="UTF-8"?>
      <Workflow id="0">
      <Start x="100" y="50" id="start"><Parameters><Parameter id="p_value"/></Parameters></Start>
      <Error x="1000" y="350"
 ↪  id="error"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Error>
      <End x="1000" y="50" id="end"><Synch_factor x="920"
 ↪  y="70">1</Synch_factor><Prerequisites><Prerequisite id="on_ok" status="Complete"/><Prerequisite
 ↪  id="on_failed" status="Complete"/></Prerequisites></End>
      <Fail x="1000" y="200"
 ↪  id="fail"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Fail>
      <Steps>
      <Step x="300" y="50"
 ↪  id="check"><Action_id>1</Action_id><Step_name>Check</Step_name><Step_type>CustomRuby</Step_type><Synch_f
        <Prerequisites><Prerequisite id="start"
 ↪  status="true"/></Prerequisites><Parameters></Parameters>
        <Inputs><Input name="value"><Map_to type="parameter"
 ↪  name="value"/></Input></Inputs><Outputs></Outputs></Step>
      <Step x="600" y="0" id="on_ok"><Action_id>2</Action_id><Step_name>On
 ↪  OK</Step_name><Step_type>CustomRuby</Step_type><Synch_factor>0</Synch_factor>
        <Prerequisites><Prerequisite id="check"
 ↪  status="Complete"/></Prerequisites><Parameters></Parameters><Inputs></Inputs><Outputs></Outputs></Step>
      <Step x="600" y="150" id="on_failed"><Action_id>3</Action_id><Step_name>On
 ↪  Failed</Step_name><Step_type>CustomRuby</Step_type><Synch_factor>0</Synch_factor>
        <Prerequisites><Prerequisite id="check"
 ↪  status="Failed"/></Prerequisites><Parameters></Parameters><Inputs></Inputs><Outputs></Outputs></Step>
      </Steps>
      <Parameters><Parameter x="100" y="200"
 ↪  id="p_value"><Name>value</Name><Value></Value><Value_type>string</Value_type><Runtime_override>true</Run
```

```
      <Notes></Notes>
      </Workflow>
- CustomRuby__1: !ruby/object:CustomRuby
    attributes:
      name: W0__Check
      optional_inputs: "{'value'=>'string'}"
      execute_code: "@status = @inputs['value'].to_s.empty? ? 'Failed' : 'Complete'"
  CustomRuby__2: !ruby/object:CustomRuby
    attributes:
      name: W0__On OK
      execute_code: "outputs['result'] = 'value was provided'"
      typed_outputs: "{'result'=>'string'}"
  CustomRuby__3: !ruby/object:CustomRuby
    attributes:
      name: W0__On Failed
      execute_code: "outputs['result'] = 'value was empty'"
      typed_outputs: "{'result'=>'string'}"
- CustomRuby__1: []
  CustomRuby__2: []
  CustomRuby__3: []
- {}
- CustomRuby: 0.7.0
```

With `{"value":"x"}`, only `On OK` runs; with `{}`, only `On Failed` runs. Both work orders end `Complete`.

10.4 Recipe: wait for a file

The workflow waits for a file matching `/tmp/ascli_watch_*.txt` on the Orchestrator host, deletes it, and returns its path.

```
---
- !ruby/object:Workflow
  attributes:
    name: wait_for_file
    run_as: SYSTEM
- !ruby/object:Whiteboard
  attributes:
    error_free: true
    xml_export: |
      <?xml version="1.0" encoding="UTF-8"?>
      <Workflow id="0">
      <Start x="100" y="50" id="start"><Parameters></Parameters></Start>
      <Error x="900" y="350"
↪   id="error"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Error>
      <End x="900" y="50" id="end"><Synch_factor>0</Synch_factor><Prerequisites><Prerequisite
↪   id="wait" status="Complete"/></Prerequisites></End>
      <Fail x="900" y="200"
↪   id="fail"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Fail>
      <Steps>
      <Step x="400" y="50" id="wait"><Action_id>1</Action_id><Step_name>Wait for
↪   file</Step_name><Step_type>LocalFileWatcher</Step_type><Synch_factor>0</Synch_factor>
        <Prerequisites><Prerequisite id="start"
↪   status="true"/></Prerequisites><Parameters></Parameters><Inputs></Inputs><Outputs></Outputs></Step>
      </Steps>
      <Parameters></Parameters><Notes></Notes>
      </Workflow>
- LocalFileWatcher__1: !ruby/object:LocalFileWatcher
    attributes:
      name: W0__Wait for file
      directory: /tmp/
      file_name: ascli_watch_*.txt
      allow_multiple: 0
      remove_file: 1
      cool_off: 2
- LocalFileWatcher__1: []
```

```
- {}
- LocalFileWatcher: 1.9.3
```

```
ascli orchestrator workflows start <id> @json:'{}' @json:'{"step":"Wait for
↪  file","variable":"FileName"}'
```

The command returns when a matching file has been stable for `cool_off` seconds, for example `/tmp/ascli_watch_20261006_230726.txt`.

10.5 Recipe: sub-workflow with outputs

Workflow `child` upper-cases its runtime parameter `text`: step `Upper`, output `result`. Workflow `parent` runs it in step `Call child` (`SubWorkflow`), and gets `result` of `Upper` as output `upper` of the step.

Import and publish `child` first, and write its id in `workflow_id` of the `SubWorkflow` template (see Referring to existing entities). The inputs and outputs of the step come from the published `child`:

- One input per runtime parameter of `child`, with the same name: here `text`, required because `Runtime_optional` is `false` in `child`.
- One output per entry of `workflow_outputs`, `{'<output name>'=>'<step name>:<output name in child>'}`, with the type of that output in `child`.

```
---
- !ruby/object:Workflow
  attributes:
    name: child
    run_as: SYSTEM
- !ruby/object:Whiteboard
  attributes:
    error_free: true
    xml_export: |
      <?xml version="1.0" encoding="UTF-8"?>
      <Workflow id="0">
      <Start x="100" y="50" id="start"><Parameters><Parameter id="p_text"/></Parameters></Start>
      <Error x="600" y="350"
↪   id="error"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Error>
      <End x="600" y="50" id="end"><Synch_factor>0</Synch_factor><Prerequisites><Prerequisite
↪   id="upper" status="Complete"/></Prerequisites></End>
      <Fail x="600" y="200"
↪   id="fail"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Fail>
      <Steps>
      <Step x="300" y="50"
↪   id="upper"><Action_id>1</Action_id><Step_name>Upper</Step_name><Step_type>CustomRuby</Step_type><Synch_
        <Prerequisites><Prerequisite id="start"
↪   status="true"/></Prerequisites><Parameters></Parameters>
        <Inputs><Input name="text"><Map_to type="parameter"
↪   name="text"/></Input></Inputs><Outputs></Outputs></Step>
      </Steps>
      <Parameters><Parameter x="100" y="200"
↪   id="p_text"><Name>text</Name><Value></Value><Value_type>string</Value_type><Runtime_override>true</Runti
      <Notes></Notes>
      </Workflow>
- CustomRuby__1: !ruby/object:CustomRuby
    attributes:
      name: W0__Upper
      mandatory_inputs: "{'text'=>'string'}"
      execute_code: "outputs['result'] = @inputs['text'].upcase"
      typed_outputs: "{'result'=>'string'}"
- CustomRuby__1: []
- {}
- CustomRuby: 0.7.0
```

```
---
- !ruby/object:Workflow
  attributes:
```

```
    name: parent
    run_as: SYSTEM
- !ruby/object:Whiteboard
  attributes:
    error_free: true
    xml_export: |
      <?xml version="1.0" encoding="UTF-8"?>
      <Workflow id="0">
      <Start x="100" y="50" id="start"><Parameters><Parameter id="p_text"/></Parameters></Start>
      <Error x="900" y="350"
↪   id="error"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Error>
      <End x="900" y="50" id="end"><Synch_factor>0</Synch_factor><Prerequisites><Prerequisite
↪   id="use" status="Complete"/></Prerequisites></End>
      <Fail x="900" y="200" id="fail"><Synch_factor>0</Synch_factor><Prerequisites><Prerequisite
↪   id="call" status="Failed"/></Prerequisites></Fail>
      <Steps>
      <Step x="300" y="50" id="call"><Action_id>1</Action_id><Step_name>Call
↪   child</Step_name><Step_type>SubWorkflow</Step_type><Synch_factor>0</Synch_factor>
        <Prerequisites><Prerequisite id="start"
↪   status="true"/></Prerequisites><Parameters></Parameters>
        <Inputs><Input name="text"><Map_to type="parameter"
↪   name="text"/></Input></Inputs><Outputs></Outputs></Step>
      <Step x="600" y="50" id="use"><Action_id>2</Action_id><Step_name>Use
↪   result</Step_name><Step_type>CustomRuby</Step_type><Synch_factor>0</Synch_factor>
        <Prerequisites><Prerequisite id="call"
↪   status="Complete"/></Prerequisites><Parameters></Parameters>
        <Inputs><Input name="value"><Map_to type="output" name="upper" step_type="SubWorkflow"
↪   action_id="1" step_name="Call child"/></Input></Inputs><Outputs></Outputs></Step>
      </Steps>
      <Parameters><Parameter x="100" y="200"
↪   id="p_text"><Name>text</Name><Value>hello</Value><Value_type>string</Value_type><Runtime_override>true</
      <Notes></Notes>
      </Workflow>
- SubWorkflow__1: !ruby/object:SubWorkflow
    attributes:
      name: W0__Call child
      workflow_id: <id of child>
      workflow_outputs: "{'upper'=>'Upper:result'}"
  CustomRuby__2: !ruby/object:CustomRuby
    attributes:
      name: W0__Use result
      mandatory_inputs: "{'value'=>'string'}"
      execute_code: "outputs['result'] = \"child returned #{@inputs['value']}\""
      typed_outputs: "{'result'=>'string'}"
- SubWorkflow__1: []
  CustomRuby__2: []
- {}
- SubWorkflow: 1.3.1
  CustomRuby: 0.7.0
```

```
ascli orchestrator workflows import child.yml @json:'{"publish":true}'
ascli orchestrator workflows import parent.yml @json:'{"publish":true}'
ascli orchestrator workflows start <id of parent> @json:'{"text":"abc"}' @json:'{"step":"Use
↪   result","variable":"result"}'   # child returned ABC
```

`child` runs in a work order of its own. The import of `parent` sets `workflow_portable_id` of the template to the portable id of `child`.

10.6 Recipe: one work order per file

Workflow `watch_folder` watches the folder `/tmp/doc_test_watch/` of the Orchestrator host, and starts workflow `per_file` once per file found:

- `Detect files` (`LocalFileWatcher`): continuous monitoring (`allow_multiple`, see Triggers), all files found at once (`return_all`): output `FileName` is an array. The files are left in the folder and remembered (`remove_file` `0`):

`per_file` deletes them.

- `Each file` (`ArrayFanout`): runs the next step once per element of input `all_files`, with the element as output `file`.
- `Process file` (`WorkflowLauncher`): starts `per_file` with `file_path` set to `file`, waits for its end (`wait_for_completion`), and ends with `Failed` if it fails (`fail_if_failure`).
- `Wait all` (`Funnelin`): runs once all executions of `Process file` have ended, and gathers their values.

`per_file` reads the size of the file and deletes it.

```
---
- !ruby/object:Workflow
  attributes:
    name: per_file
    run_as: SYSTEM
- !ruby/object:Whiteboard
  attributes:
    error_free: true
    xml_export: |
      <?xml version="1.0" encoding="UTF-8"?>
      <Workflow id="0">
      <Start x="100" y="50" id="start"><Parameters><Parameter id="p_file_path"/></Parameters></Start>
      <Error x="600" y="350"
↪   id="error"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Error>
      <End x="600" y="50" id="end"><Synch_factor>0</Synch_factor><Prerequisites><Prerequisite
↪   id="process" status="Complete"/></Prerequisites></End>
      <Fail x="600" y="200"
↪   id="fail"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Fail>
      <Steps>
      <Step x="300" y="50" id="process"><Action_id>1</Action_id><Step_name>Process
↪   file</Step_name><Step_type>CustomRuby</Step_type><Synch_factor>0</Synch_factor>
        <Prerequisites><Prerequisite id="start"
↪   status="true"/></Prerequisites><Parameters></Parameters>
        <Inputs><Input name="file_path"><Map_to type="parameter"
↪   name="file_path"/></Input></Inputs><Outputs></Outputs></Step>
      </Steps>
      <Parameters><Parameter x="100" y="200"
↪   id="p_file_path"><Name>file_path</Name><Value></Value><Value_type>string</Value_type><Runtime_override>t
      <Notes></Notes>
      </Workflow>
- CustomRuby__1: !ruby/object:CustomRuby
    attributes:
      name: W0__Process file
      mandatory_inputs: "{'file_path'=>'string'}"
      execute_code: "outputs['size'] = File.size(@inputs['file_path']);
↪   File.delete(@inputs['file_path'])"
      typed_outputs: "{'size'=>'int'}"
- CustomRuby__1: []
- {}
- CustomRuby: 0.7.3
```

```
---
- !ruby/object:Workflow
  attributes:
    name: watch_folder
    run_as: SYSTEM
- !ruby/object:Whiteboard
  attributes:
    error_free: true
    xml_export: |
      <?xml version="1.0" encoding="UTF-8"?>
      <Workflow id="0">
      <Start x="100" y="50" id="start"><Parameters></Parameters></Start>
      <Error x="1200" y="350"
↪   id="error"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Error>
      <End x="1200" y="50" id="end"><Synch_factor>0</Synch_factor><Prerequisites><Prerequisite
↪   id="wait" status="Complete"/></Prerequisites></End>
```

```
      <Fail x="1200" y="200" id="fail"><Synch_factor>0</Synch_factor><Prerequisites><Prerequisite
↪  id="wait" status="Failed"/></Prerequisites></Fail>
      <Steps>
      <Step x="300" y="50" id="detect"><Action_id>1</Action_id><Step_name>Detect
↪  files</Step_name><Step_type>LocalFileWatcher</Step_type><Synch_factor>0</Synch_factor>
        <Prerequisites><Prerequisite id="start"
↪  status="true"/></Prerequisites><Parameters></Parameters><Inputs></Inputs><Outputs></Outputs></Step>
      <Step x="525" y="50" id="each"><Action_id>2</Action_id><Step_name>Each
↪  file</Step_name><Step_type>ArrayFanout</Step_type><Synch_factor>0</Synch_factor>
        <Prerequisites><Prerequisite id="detect"
↪  status="Complete"/></Prerequisites><Parameters></Parameters>
        <Inputs><Input name="all_files"><Map_to type="output" name="FileName"
↪  step_type="LocalFileWatcher" action_id="1" step_name="Detect
↪  files"/></Input></Inputs><Outputs></Outputs></Step>
      <Step x="750" y="50" id="launch"><Action_id>3</Action_id><Step_name>Process
↪  file</Step_name><Step_type>WorkflowLauncher</Step_type><Synch_factor>0</Synch_factor>
        <Prerequisites><Prerequisite id="each"
↪  status="Complete"/></Prerequisites><Parameters></Parameters>
        <Inputs><Input name="file_path"><Map_to type="output" name="file" step_type="ArrayFanout"
↪  action_id="2" step_name="Each file"/></Input></Inputs><Outputs></Outputs></Step>
      <Step x="975" y="50" id="wait"><Action_id>4</Action_id><Step_name>Wait
↪  all</Step_name><Step_type>Funnelin</Step_type><Synch_factor x="895" y="70">1</Synch_factor>
        <Prerequisites><Prerequisite id="launch" status="Complete"/><Prerequisite id="launch"
↪  status="Failed"/><Prerequisite id="launch"
↪  status="Error"/></Prerequisites><Parameters></Parameters>
        <Inputs><Input name="file"><Map_to type="output" name="file" step_type="ArrayFanout"
↪  action_id="2" step_name="Each file"/></Input>
        <Input name="WorkOrder_ID"><Map_to type="output" name="WorkOrder_ID"
↪  step_type="WorkflowLauncher" action_id="3" step_name="Process
↪  file"/></Input></Inputs><Outputs></Outputs></Step>
      </Steps>
      <Parameters></Parameters><Notes></Notes>
      </Workflow>
- LocalFileWatcher__1: !ruby/object:LocalFileWatcher
    attributes:
      name: W0__Detect files
      directory: /tmp/doc_test_watch/
      file_name: "*.txt"
      allow_multiple: 1
      remove_file: 0
      return_all: 1
      cool_off: 2
  ArrayFanout__2: !ruby/object:ArrayFanout
    attributes:
      name: W0__Each file
      fanout_variable: all_files
      fanned_as: file
      variable_type: string
  WorkflowLauncher__3: !ruby/object:WorkflowLauncher
    attributes:
      name: W0__Process file
      workflow_id: <id of per_file>
      wait_for_completion: 1
      fail_if_failure: 1
  Funnelin__4: !ruby/object:Funnelin
    attributes:
      name: W0__Wait all
      funnel_variable: file
      funnel_type: string
      funneled_inputs: "{'WorkOrder_ID'=>'int'}"
      return_status: 1
      fail_if_failures: 1
- LocalFileWatcher__1: []
  ArrayFanout__2: []
  WorkflowLauncher__3: []
  Funnelin__4: []
```

```
- {}
- LocalFileWatcher: 1.9.3
  ArrayFanout: 1.2.2
  WorkflowLauncher: 1.6.1
  Funnelin: 1.1.3
```

```
ascli orchestrator workflows import per_file.yml @json:'{"publish":true}'
ascli orchestrator workflows import watch_folder.yml @json:'{"publish":true}'
ascli orchestrator workflows start <id of watch_folder> @json:'{}'
```

Observed with the files `a_ok.txt`, `b_ok.txt` and `c_locked.txt`, the last one made impossible to delete (`chattr +i`). The folder belongs to the user that runs Orchestrator (`aspweb`), so that `per_file` can delete the files:

- One work order of `per_file` per file. The executions of the fanned step are named `Process file (0)`, `Process file (1)`, ... in `Step_information`.
- Outputs of `Wait all`, by work step id: `Hash_by_state_id` (value of `file`), `Status_by_state_id`; by value of `file`: `Status_hash`, and `WorkOrder_ID_hash` for the input `WorkOrder_ID` declared in `funneled_inputs`. For example `Status_hash` is `{"/tmp/doc_test_watch/a_ok.txt" => "Complete", "/tmp/doc_test_watch/b_ok.txt" => "Complete", "/tmp/doc_test_watch/c_locked.txt" => "Failed"}`.
- `Wait all` ends with `Failed` (`fail_if_failures`: `Funneled in 3 work steps with 1 failures`), and the work order goes to `Fail`. When all files are processed, it goes to `End`.
- The work order that detected the files continues; a clone waits for the next files: cancel it to stop the monitoring.
- `c_locked.txt` stays in the folder, and the clone does not detect it again: it shares the history of the first work order.

Rules of this pattern (from `ArrayFanout` and `Funnelin`, and tests):

- Exactly one step between `ArrayFanout` and `Funnelin`. `ArrayFanout` ends with `Error` when its `Complete` exit leads to more than one step ("Only one step is supported between fan-out and funnel-in"), or when the `Complete` exit of that step leads to more than one step ("Funneled-step should only exit complete in the funnel-in step"). A chain of two steps is not refused, but is wrong: observed with `ArrayFanout`, two `FileInfo` steps and `Funnelin`, on three files, the first step ran three times, the second one once (first file), and `Funnelin` reported `Funneled in 1 work steps`; the work order completed. For more processing, use a `SubWorkflow` or `WorkflowLauncher` step, as here.
- Connect `Funnelin` to each end status of that step (`Complete`, `Failed`, `Error`), with `Synch_factor` 1: `ArrayFanout` replaces 1 by the number of elements, so `Funnelin` waits for all executions, whatever their status. With a link on `Complete` only, a failed execution has no next step, and the work order fails at once (`No behavior defined for step status change to Failed - in Process file (1)`).
- Write `remove_file` `0` (or `1`). Left blank, the watcher keeps no history (see LocalFileWatcher): a file still in the folder is detected again by each clone. This was observed with this recipe without `remove_file`: with `c_locked.txt` left in the folder, each new clone started `per_file` on it again.
- A new start of `watch_folder` has an empty history: it processes again the files left in the folder. Move a file whose processing fails to an error folder.

10.7 Recipe: queue and resource pool

Workflow `queue` runs step `Work` for one work order at a time, in the order the work orders started:

- `Enqueue` (`QueueStager`, operation `Add`): adds the runtime parameter `item` to the queue `doc test queue`, created on first use (`adhoc_queue`). With `lifetime` `WorkOrder`, the items of a work order that stopped are removed from the queue.
- `Reserve` (`LoadManager`, action `Reserve`): reserves a resource of the pool `doc test pool`, created on first use (default `is_adhoc`) with one resource of capacity 1 (`max_resources`, `resource_capacity`). With `queue_id` set to the queue, and input `Expected_queued_item` mapped to output `Queued_item` of `Enqueue`, it waits until this item is the first of the queue and a resource is free, and then removes the item from the queue. The value of an item identifies it: use a value that is unique to the work order, for example a file path.
- `Work` (`CustomRuby`): 10 seconds of work; with `fail` set to `yes`, it raises an exception (`Error`).
- `Release` and `Release on error` (`LoadManager`, action `Release`): release the reservation (input `Allocation_token_to_release` mapped to output `Allocation_token` of `Reserve`), after `Complete` of `Work`, and after `Failed`

or `Error` (`Synch_factor` 1). A reservation that is not released holds the resource until `resource_hold_timeout` (none by default): the next work orders wait.

```
---
- !ruby/object:Workflow
  attributes:
    name: queue
    run_as: SYSTEM
- !ruby/object:Whiteboard
  attributes:
    error_free: true
    xml_export: |
      <?xml version="1.0" encoding="UTF-8"?>
      <Workflow id="0">
      <Start x="100" y="50" id="start"><Parameters><Parameter id="p_item"/><Parameter
↪   id="p_fail"/></Parameters></Start>
      <Error x="1200" y="350"
↪   id="error"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Error>
      <End x="1200" y="50" id="end"><Synch_factor>0</Synch_factor><Prerequisites><Prerequisite
↪   id="release" status="Complete"/></Prerequisites></End>
      <Fail x="1200" y="200" id="fail"><Synch_factor>0</Synch_factor><Prerequisites><Prerequisite
↪   id="release_error" status="Complete"/></Prerequisites></Fail>
      <Steps>
      <Step x="300" y="50"
↪   id="enqueue"><Action_id>1</Action_id><Step_name>Enqueue</Step_name><Step_type>QueueStager</Step_type><Sy
        <Prerequisites><Prerequisite id="start"
↪   status="true"/></Prerequisites><Parameters></Parameters>
        <Inputs><Input name="Queued_item"><Map_to type="parameter"
↪   name="item"/></Input></Inputs><Outputs></Outputs></Step>
      <Step x="525" y="50"
↪   id="reserve"><Action_id>2</Action_id><Step_name>Reserve</Step_name><Step_type>LoadManager</Step_type><Sy
        <Prerequisites><Prerequisite id="enqueue"
↪   status="Complete"/></Prerequisites><Parameters></Parameters>
        <Inputs><Input name="Expected_queued_item"><Map_to type="output" name="Queued_item"
↪   step_type="QueueStager" action_id="1"
↪   step_name="Enqueue"/></Input></Inputs><Outputs></Outputs></Step>
      <Step x="750" y="50"
↪   id="work"><Action_id>3</Action_id><Step_name>Work</Step_name><Step_type>CustomRuby</Step_type><Synch_fac
        <Prerequisites><Prerequisite id="reserve"
↪   status="Complete"/></Prerequisites><Parameters></Parameters>
        <Inputs><Input name="fail"><Map_to type="parameter"
↪   name="fail"/></Input></Inputs><Outputs></Outputs></Step>
      <Step x="975" y="50"
↪   id="release"><Action_id>4</Action_id><Step_name>Release</Step_name><Step_type>LoadManager</Step_type><Sy
        <Prerequisites><Prerequisite id="work"
↪   status="Complete"/></Prerequisites><Parameters></Parameters>
        <Inputs><Input name="Allocation_token_to_release"><Map_to type="output"
↪   name="Allocation_token" step_type="LoadManager" action_id="2"
↪   step_name="Reserve"/></Input></Inputs><Outputs></Outputs></Step>
      <Step x="975" y="200" id="release_error"><Action_id>5</Action_id><Step_name>Release on
↪   error</Step_name><Step_type>LoadManager</Step_type><Synch_factor x="895" y="220">1</Synch_factor>
        <Prerequisites><Prerequisite id="work" status="Failed"/><Prerequisite id="work"
↪   status="Error"/></Prerequisites><Parameters></Parameters>
        <Inputs><Input name="Allocation_token_to_release"><Map_to type="output"
↪   name="Allocation_token" step_type="LoadManager" action_id="2"
↪   step_name="Reserve"/></Input></Inputs><Outputs></Outputs></Step>
      </Steps>
      <Parameters><Parameter x="100" y="200"
↪   id="p_item"><Name>item</Name><Value></Value><Value_type>string</Value_type><Runtime_override>true</Runti
      <Parameter x="100" y="300"
↪   id="p_fail"><Name>fail</Name><Value>no</Value><Value_type>string</Value_type><Runtime_override>true</Run
      <Notes></Notes>
      </Workflow>
- QueueStager__1: !ruby/object:QueueStager
    attributes:
      name: W0__Enqueue
      operation: Add
```

```
      queue_name: doc test queue
      adhoc_queue: 1
      lifetime: WorkOrder
  LoadManager__2: !ruby/object:LoadManager
    attributes:
      name: W0__Reserve
      manager_action: Reserve
      unique_id: doc test pool
      max_resources: 1
      resource_capacity: 1
      unicity_type: absolute
      queue_id: doc test queue
  CustomRuby__3: !ruby/object:CustomRuby
    attributes:
      name: W0__Work
      optional_inputs: "{'fail'=>'string'}"
      execute_code: "sleep 10; raise 'asked to fail' if @inputs['fail'] == 'yes'; outputs['done_at']
↪  = Time.now.utc.strftime('%H:%M:%S')"
      typed_outputs: "{'done_at'=>'string'}"
  LoadManager__4: !ruby/object:LoadManager
    attributes:
      name: W0__Release
      manager_action: Release
  LoadManager__5: !ruby/object:LoadManager
    attributes:
      name: W0__Release on error
      manager_action: Release
- QueueStager__1: []
  LoadManager__2: []
  CustomRuby__3: []
  LoadManager__4: []
  LoadManager__5: []
- {}
- QueueStager: 0.9.5
  LoadManager: 0.5.3
  CustomRuby: 0.7.3
```

```
ascli orchestrator workflows import queue.yml @json:'{"publish":true}'
ascli orchestrator workflows start <id> @json:'{"item":"a"}'
ascli orchestrator workflows start <id> @json:'{"item":"b","fail":"yes"}'
ascli orchestrator workflows start <id> @json:'{"item":"c"}'
ascli orchestrator queues list
ascli orchestrator pools status 'doc test pool'   # taken 0, capacity 1: all released
```

Observed: `Reserve` waited 0 s for `a`, 15 s for `b` and 25 s for `c` (`processingTime` in `Step_information`). `a` and `c` end `Complete`; `b` ends `Failed`, through `Release on error`. Generic resources are named `resource_1`, `resource_2`, … (output `Resource_reserved`).

10.8 Recipe: lookup table

Workflow `lookup` takes the settings of the runtime parameter `profile` from a table (see SimpleParameterLookup), and fails for an unknown profile. Rows are separated by `\n` in a double-quoted value.

```
---
- !ruby/object:Workflow
  attributes:
    name: lookup
    run_as: SYSTEM
- !ruby/object:Whiteboard
  attributes:
    error_free: true
    xml_export: |
      <?xml version="1.0" encoding="UTF-8"?>
      <Workflow id="0">
      <Start x="100" y="50" id="start"><Parameters><Parameter id="p_profile"/></Parameters></Start>
```

```
      <Error x="900" y="350"
↪  id="error"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Error>
      <End x="900" y="50" id="end"><Synch_factor>0</Synch_factor><Prerequisites><Prerequisite
↪  id="build" status="Complete"/></Prerequisites></End>
      <Fail x="900" y="200" id="fail"><Synch_factor>0</Synch_factor><Prerequisites><Prerequisite
↪  id="lookup" status="Failed"/></Prerequisites></Fail>
      <Steps>
      <Step x="300" y="50" id="lookup"><Action_id>1</Action_id><Step_name>Profile
↪  settings</Step_name><Step_type>SimpleParameterLookup</Step_type><Synch_factor>0</Synch_factor>
        <Prerequisites><Prerequisite id="start"
↪  status="true"/></Prerequisites><Parameters></Parameters>
        <Inputs><Input name="Profile"><Map_to type="parameter"
↪  name="profile"/></Input></Inputs><Outputs></Outputs></Step>
      <Step x="600" y="50" id="build"><Action_id>2</Action_id><Step_name>Build
↪  command</Step_name><Step_type>CustomRuby</Step_type><Synch_factor>0</Synch_factor>
        <Prerequisites><Prerequisite id="lookup"
↪  status="Complete"/></Prerequisites><Parameters></Parameters>
        <Inputs><Input name="options"><Map_to type="output" name="Options"
↪  step_type="SimpleParameterLookup" action_id="1" step_name="Profile settings"/></Input>
        <Input name="extension"><Map_to type="output" name="Extension"
↪  step_type="SimpleParameterLookup" action_id="1" step_name="Profile
↪  settings"/></Input></Inputs><Outputs></Outputs></Step>
      </Steps>
      <Parameters><Parameter x="100" y="200"
↪  id="p_profile"><Name>profile</Name><Value>MP4</Value><Value_type>string</Value_type><Runtime_override>t
      <Notes></Notes>
      </Workflow>
- SimpleParameterLookup__1: !ruby/object:SimpleParameterLookup
    attributes:
      name: W0__Profile settings
      parameters_associations: "Profile:::Options:::Extension\nMP4:::-c:v libx264:::mp4\nWEBM:::-c:v
↪  libvpx-vp9:::webm\n/^AUDIO/:::-vn -c:a libmp3lame:::mp3"
  CustomRuby__2: !ruby/object:CustomRuby
    attributes:
      name: W0__Build command
      mandatory_inputs: "{'options'=>'string','extension'=>'string'}"
      execute_code: "outputs['command'] = \"ffmpeg -i in.mov #{@inputs['options']}
↪  out.#{@inputs['extension']}\""
      typed_outputs: "{'command'=>'string'}"
- SimpleParameterLookup__1: []
  CustomRuby__2: []
- {}
- SimpleParameterLookup: 0.8.8
  CustomRuby: 0.7.3
```

```
ascli orchestrator workflows start <id> @json:'{"profile":"webm"}' @json:'{"step":"Build
↪  command","variable":"command"}'       # ffmpeg -i in.mov -c:v libvpx-vp9 out.webm
ascli orchestrator workflows start <id> @json:'{"profile":"AUDIO_HQ"}' @json:'{"step":"Build
↪  command","variable":"command"}'   # ffmpeg -i in.mov -vn -c:a libmp3lame out.mp3
```

- `webm` matches `WEBM`: the comparison ignores case by default (`case_sensitive`).
- `AUDIO_HQ` matches the row `/^AUDIO/`, a regular expression.
- With `{}`, `profile` has its default value `MP4`.
- `PRORES` matches no row: `Profile settings` ends with `Failed` (`No configuration match for data set: Profile:PRORES`), and the work order goes to `Fail`.

10.9 Recipe: operator decision

Workflow `review` delivers to the runtime parameter `target` if it is valid, and otherwise asks an operator for another target. Two branches join in a `MergePoint` step:

- `Check target` (`Filter`): `Complete` if `target` starts with `/data/`, `Failed` otherwise.
- `Ask operator` (`UserInput`), after `Failed`: creates a task in the web interface (tab Tasks) for the user `admin` (`assigned_user_logins`), described as `Invalid target for <title>`. The task shows `title` and `target`

(`mandatory_inputs`), and asks for `new_target` (required, `required_requested_data`) and `rejected` (optional, `user_outputs`): they become outputs of the step. For another layout of the task, see Custom form.

- `Check rejection` (`Filter`): `Failed` if the operator wrote `yes` in `rejected`: the work order goes to `Fail`.
- `Merge target` (`MergePoint`), after `Complete` of `Check target` or of `Check rejection` (`Synch_factor` 1): output `target` is input `target` (mapped to `new_target` of `Ask operator`) when it has a value, and input `target_alternate` (mapped to the parameter `target`) otherwise. When `Ask operator` did not run, its outputs have no value.
- `Deliver` (`CustomRuby`): uses `target` of `Merge target`.

```
---
- !ruby/object:Workflow
  attributes:
    name: review
    run_as: SYSTEM
- !ruby/object:Whiteboard
  attributes:
    error_free: true
    xml_export: |
      <?xml version="1.0" encoding="UTF-8"?>
      <Workflow id="0">
      <Start x="100" y="50" id="start"><Parameters><Parameter id="p_title"/><Parameter
↪   id="p_target"/></Parameters></Start>
      <Error x="1400" y="350"
↪   id="error"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Error>
      <End x="1400" y="50" id="end"><Synch_factor>0</Synch_factor><Prerequisites><Prerequisite
↪   id="deliver" status="Complete"/></Prerequisites></End>
      <Fail x="1400" y="200" id="fail"><Synch_factor>0</Synch_factor><Prerequisites><Prerequisite
↪   id="check_rejection" status="Failed"/></Prerequisites></Fail>
      <Steps>
      <Step x="300" y="50" id="check_target"><Action_id>1</Action_id><Step_name>Check
↪   target</Step_name><Step_type>Filter</Step_type><Synch_factor>0</Synch_factor>
        <Prerequisites><Prerequisite id="start"
↪   status="true"/></Prerequisites><Parameters></Parameters>
        <Inputs><Input name="target"><Map_to type="parameter"
↪   name="target"/></Input></Inputs><Outputs></Outputs></Step>
      <Step x="500" y="200" id="ask"><Action_id>2</Action_id><Step_name>Ask
↪   operator</Step_name><Step_type>UserInput</Step_type><Synch_factor>0</Synch_factor>
        <Prerequisites><Prerequisite id="check_target"
↪   status="Failed"/></Prerequisites><Parameters></Parameters>
        <Inputs><Input name="title"><Map_to type="parameter" name="title"/></Input><Input
↪   name="target"><Map_to type="parameter" name="target"/></Input></Inputs><Outputs></Outputs></Step>
      <Step x="750" y="200" id="check_rejection"><Action_id>3</Action_id><Step_name>Check
↪   rejection</Step_name><Step_type>Filter</Step_type><Synch_factor>0</Synch_factor>
        <Prerequisites><Prerequisite id="ask"
↪   status="Complete"/></Prerequisites><Parameters></Parameters>
        <Inputs><Input name="rejected"><Map_to type="output" name="rejected" step_type="UserInput"
↪   action_id="2" step_name="Ask operator"/></Input></Inputs><Outputs></Outputs></Step>
      <Step x="1000" y="50" id="merge"><Action_id>4</Action_id><Step_name>Merge
↪   target</Step_name><Step_type>MergePoint</Step_type><Synch_factor x="920" y="70">1</Synch_factor>
        <Prerequisites><Prerequisite id="check_target" status="Complete"/><Prerequisite
↪   id="check_rejection" status="Complete"/></Prerequisites><Parameters></Parameters>
        <Inputs><Input name="target"><Map_to type="output" name="new_target" step_type="UserInput"
↪   action_id="2" step_name="Ask operator"/></Input>
        <Input name="target_alternate"><Map_to type="parameter"
↪   name="target"/></Input></Inputs><Outputs></Outputs></Step>
      <Step x="1200" y="50"
↪   id="deliver"><Action_id>5</Action_id><Step_name>Deliver</Step_name><Step_type>CustomRuby</Step_type><Syn
        <Prerequisites><Prerequisite id="merge"
↪   status="Complete"/></Prerequisites><Parameters></Parameters>
        <Inputs><Input name="target"><Map_to type="output" name="target" step_type="MergePoint"
↪   action_id="4" step_name="Merge target"/></Input></Inputs><Outputs></Outputs></Step>
      </Steps>
      <Parameters><Parameter x="100" y="200" id="p_title"><Name>title</Name><Value>clip
↪   1</Value><Value_type>string</Value_type><Runtime_override>true</Runtime_override><Runtime_optional>true<
```

```
      <Parameter x="100" y="300"
↪   id="p_target"><Name>target</Name><Value>/data/default</Value><Value_type>string</Value_type><Runtime_ove
      <Notes></Notes>
      </Workflow>
- Filter__1: !ruby/object:Filter
    attributes:
      name: W0__Check target
      command: "'<%= target %>'.start_with?('/data/')"
  UserInput__2: !ruby/object:UserInput
    attributes:
      name: W0__Ask operator
      event_description: "Invalid target for <%= title %>"
      assigned_user_logins: admin
      mandatory_inputs: "{'title'=>'string','target'=>'string'}"
      required_requested_data: "{'new_target'=>'string'}"
      user_outputs: "{'rejected'=>'string'}"
  Filter__3: !ruby/object:Filter
    attributes:
      name: W0__Check rejection
      command: "'<%= rejected %>' != 'yes'"
  MergePoint__4: !ruby/object:MergePoint
    attributes:
      name: W0__Merge target
      merged_inputs: "{'target'=>['','','string','required']}"
  CustomRuby__5: !ruby/object:CustomRuby
    attributes:
      name: W0__Deliver
      mandatory_inputs: "{'target'=>'string'}"
      execute_code: "outputs['result'] = \"delivered to #{@inputs['target']}\""
      typed_outputs: "{'result'=>'string'}"
- Filter__1: []
  UserInput__2: []
  Filter__3: []
  MergePoint__4: []
  CustomRuby__5: []
- {}
- Filter: 0.3.1
  UserInput: 0.8.2
  MergePoint: 0.5.2
  CustomRuby: 0.7.3
```

```
ascli orchestrator workflows start <id> @json:'{"target":"/data/ok"}'
↪  @json:'{"step":"Deliver","variable":"result"}'   # delivered to /data/ok
ascli orchestrator workflows start <id> @json:'{"title":"clip 2","target":"bad/path"}'
```

The second work order waits on `Ask operator`. In the web interface, the operator fills the form and clicks `Complete Task`. Observed:

- `new_target` `/data/fixed`, `rejected` empty: `delivered to /data/fixed`, work order `Complete`.
- `rejected` `yes`: `Check rejection` ends with `Failed`, work order `Failed`.
- Output `InputProvider` of `Ask operator` is the id of the user who completed the task (`1` for `admin`).

The REST API can also complete a task:

```
GET api/list_tasks/self                                          # the tasks of the current user: id,
↪  event, requested_user_inputs (name, required, value_type)
GET api/task_details/<TASK_ID>                                   # the task, and the values shown to
↪  the operator in task_inputs
GET api/submit_task/<TASK_ID>?outputs[new_target]=/data/fixed    # completes the task
```

Observed on 4.1.6:

- Without a required output, `submit_task` refuses with `Missing data for <NAME>` (HTTP 404, in the error description), and the task stays open.
- A step assigned to several groups has one task per group: when one is completed, the others end with status `Preempted`.

10.9.1 Custom form

`Ask operator` above uses the default form: the inputs, then one field per requested data. For another layout, write the content of the form in a file, and give its name in attribute `view_template`. The file replaces the default content. The buttons `Cancel` and `Complete Task` stay below it, in the same HTML form: do not add them.

Install the file on the Orchestrator server, with owner `aspweb`: `/opt/aspera/var/config/orchestrator/actions_config/user_inputs/_<NAME>.html.erb`. The name must start with `_`. The designer lists the files of this folder in View template.

In the file	Meaning
`received_parameters['<NAME>']`	Value of the input `<NAME>` of the step.
`requested_user_inputs`	The requested data: a map of name to type.
`active_assignment`	The task.
A field named `outputs[<NAME>]`	Value of the output `<NAME>` of the step.
`draw_input('outputs', '<NAME>', '<TYPE>', <DEFAULT_VALUE>, '<HTML_ATTRIBUTES>')`	Draws a field for the type: `flag`, `int`, `float`, `string`, `pwd`, `array`, `hash`, `object`, `date` or `attachment`. The last two arguments are optional.

This form shows a file name and a video, and asks for a target, a path, a date with time, and a description. It is a shorter version of the form in the figure of the Development Practices.

`_external_delivery.html.erb`:

```
<%# Inside the task form: the buttons Cancel and Complete Task are already there. %>
<style>
  .delivery .bar { background: #00608f; color: #fff; padding: 0.15em 0.7em; }
  .delivery video { width: 100%; margin: 0.6em 0; }
  .delivery p { margin: 0.6em 0; }
  .delivery label { margin-right: 2em; }
</style>
<div class="delivery">
  <h3>External delivery</h3>
  <div class="bar">File name</div>
  <p><%= File.basename(received_parameters['media_file_path'].to_s) %></p>
  <div class="bar">Preview</div>
  <video controls preload="metadata" src="https://test-videos.co.
   ↪ uk/vids/bigbuckbunny/mp4/h264/360/Big_Buck_Bunny_360_10s_1MB.mp4"></video>
  <div class="bar">Target</div>
  <p>
    <% ['VOD Distrib', 'CABLE Distrib', 'MOBILE Distrib'].each do |target| %>
      <label><input type="radio" name="outputs[target]" value="<%= target %>" required> <%= target
       ↪ %></label>
    <% end %>
  </p>
  <p><label>Media path <input type="text" name="outputs[final_media_path]"
   ↪ value="/export/cable/hi-res/"></label></p>
  <p><label>Offer start <input type="datetime-local" name="outputs[date_start]"></label></p>
  <p><label>Description <%= draw_input('outputs', 'description',
   ↪ requested_user_inputs['description']) %></label></p>
</div>
```

Attributes of the `UserInput` template:

```
view_template: _external_delivery.html.erb
mandatory_inputs: "{'media_file_path'=>'string'}"
required_requested_data: "{'target'=>'string'}"
user_outputs: "{'final_media_path'=>'string','date_start'=>'string','description'=>'string'}"
```

Observed on 4.1.6:

- The file is read each time the task opens: a change applies without a restart. If the file is missing, the task does not open (HTTP 500).
- A workflow export contains only the name of the file: copy the file to the target server.
- The Ruby code runs on the server when the task opens. The task page has no Content-Security-Policy: a `<script>` in the file runs, and an external video loads.
- The video of this sample is a fixed test file. In a real form, pass the URL in an input, for example the output `output_file_path` of a `ConvertPath` step: with `input_mount_point` `/data/ingest/media` and `output_mount_point` `https://<WEB_HOST>/media/`, the file `/data/ingest/media/clip1.mp4` gives `https://<WEB_HOST>/media/clip1.mp4`. Use a web server that the browser of the operator can reach: `/aspera/orchestrator/images/` on the Orchestrator server gives HTTP 500.
- Only the outputs declared in the template are stored: a field with another name is ignored.
- A declared output with no field in the submitted form gets the text `nil`, not an empty value: for example `target` when no radio button is selected. An empty text field gives an empty value. An output not given to `api/submit_task` also gets the text `nil`.
- When the form is submitted, the server does not check the required data: without `target`, the step ends with `Complete`. Only the browser can refuse: set the HTML attribute `required`, as in this sample. `api/submit_task` does check them.
- `Cancel` only closes the dialog, and the task stays open. To let the operator refuse, ask for it in a field, such as `rejected` in the recipe above.
- `draw_input` with type `string` draws a field that is read-only until it has the focus.
- `draw_input` with type `date` has no time. For a date with time, use `<input type="datetime-local">` and requested data of type `string`: the value looks like `2026-12-01T08:00`.

10.10 Update an existing workflow

`import_workflow` always creates a new workflow. To add a revision to an existing workflow, the file must be on the Orchestrator host, and `import_with_constraints` is called with `add as revision`:

1. Have the file on the server. `import_workflow` stores each uploaded file in `<run_dir>/workflows/import/<import_file_name>` (for example `/opt/aspera/orchestrator/var/run/orchestrator/workflows/import/`), but also imports it as a new workflow. Upload it with `publish_workflow` set to `false`, so that this copy is not published, and delete the copy in the user interface.
2. Call `import_with_constraints` with the path of the file and the id of the workflow to update (see import_with_constraints). The response is the workflow; its last revision is the imported one.
3. Publish the workflow: the new revision becomes the running one.

The new revision has its own action templates: an embedded template is named `W<workflow id>__<step name>_1`, because the name `W<workflow id>__<step name>` is used by the template of the previous revision.

With `ascli`:

```
ascli orchestrator workflows import_with_constraints
 ↪  @json:'{"filename":"/opt/aspera/orchestrator/var/run/orchestrator/workflows/import/my_workflow.
 ↪  yml","add_as_revision":239}'
ascli orchestrator workflows publish 239
```

Chapter 11

Authoring checklist

Import and publish do not check these rules. The designer checks most of them (`Whiteboard#parse_workflow`) before setting `error_free`.

File:

- Six elements, in order; elements 4, 5 and 6 present, even empty.
- Plain format: no line matching `concise_attributes:`; booleans of templates written `1` and `0`.
- `error_free: true` in the whiteboard.
- Each template key `<StepType>__<N>` matches the `Step_type` and `Action_id` of its steps.
- Embedded templates named `W0__<step name>`.
- `LocalFileWatcher` templates have `remove_file` `0` or `1`, not blank (else no trigger history).
- No secret in the file: secrets are given by runtime `pwd` parameters, or by remote nodes (see Secrets).

Graph:

- `Start`, `End`, `Fail`, `Error` present once; at least one `Step`.
- `End`, `Fail`, `Error` and each `Step` have `Synch_factor`; each `Step` has `Inputs` and `Outputs` (may be empty).
- Each step has a name, unique in the workflow, and a type of an enabled plugin.
- Each step has at least one prerequisite (else it never runs: ”Step is orphaned”).
- Each step is the prerequisite of at least one node with status `Complete` or `true` (else: ”Step does not have output”).
- At least one prerequisite reaches `End`, `Fail` or `Error` (else: ”Workflow does not terminate”).
- Each `Map_to` of type `output` refers to an existing step, by `step_name`, `step_type` and `action_id`.
- Each `Map_to` of type `parameter` refers to a defined parameter, linked to `Start` or a step.
- Each required input of each template is mapped.
- Each parameter has a `Name` and a `Value_type`; a parameter that is not a runtime parameter, or that is an optional runtime parameter, has a `Value`.
- Join nodes after mutually exclusive / alternative branches have `Synch_factor` `1`; join nodes after concurrent parallel nominal branches have `Synch_factor` 0.
- Each `Synch_factor` other than `0` has attributes `x` and `y`, else the designer shows an empty canvas.
- Each `router`, when present, is one of the four `draw2d.*ConnectionRouter` values: Manhattan on step prerequisites (Bezier only where the graph reads better), Bezier on parameter links.
- Retry settings (`Error_*`, `Failed_*`) only on steps that have an outgoing branch with that status.

Chapter 12

REST API

Base URL: `https://<host>/aspera/orchestrator/api/`. Authentication: a JWT obtained with `POST api/login`, or query parameters `login` and `password` when JWT authentication is disabled. The response format is chosen with query parameter `format` (`xml`, the default, or `json`).

Routes: each endpoint below has a route with the given method. In addition, any endpoint can be called as `api/<endpoint>/<id>`, with any HTTP method, where `<id>` is the main identifier (workflow or work order). Some endpoints have no other route (for example `export_workflow`, `list_monitors`, `journal_files`, `pool_status`): when they do not use `<id>`, any value works, for example `0`. The OpenAPI description lists all endpoints and their parameters, checked against Orchestrator with `rake doc:api_check` of this repository.

Errors are returned with the exception message and stack trace in the response body.

12.1 import_workflow

`POST api/import_workflow`, body `multipart/form-data`:

Field	Required	Content
`import_file`	yes	The workflow file (file part).
`import_file_name`	yes	File name, ending with `yml` or `wkf`. The upload is stored under this name in `<run_dir>/workflows/import/`.
`folder_id`	no	Folder of the new workflow.
`link_to_subworkflows`	no	`true`: in a `.wkf` package, use the sub-workflows already imported from the same files instead of importing them again. See Referring to existing entities.
`publish_workflow`	no	`true` or `1`: publish after import. The server option `publish_api_imported_workflow` sets the default.

Response: the new workflow as JSON (`{"workflow": {"id": ..., "name": ..., ...}}`), in both formats. Other responses with status 200:

- `{"plugins": [...], "missing_deps": [...]}`: plugins that could not be enabled; nothing is imported.

Orchestrator also has a response `{"dependencies": ..., "file": ..., "workflow": ...}` for dependencies that are not packed, but its condition is never met (see Known issues).

On error: status 500 with `{"error": ..., "message": ..., "details": ...}`.

12.2 export_workflow

`GET api/export_workflow/<id or portable id>` (no route without the id in the path)

Returns the workflow file (`.yml`, Rails format). With parameter `export_with_dependencies`, returns a `.wkf` package with sub-workflows and remote nodes. Any non-empty value of `export_with_dependencies`, including `false`, enables it.

12.3 publish_workflow

`POST api/publish_workflow?id=<id or portable id>`, or `api/publish_workflow/<id or portable id>` with any method. The id can also be given in a JSON body `{"id": ...}`.

Publishes the last whiteboard revision. Response: `true` or `false` (JSON), or `<WorkflowPublish><Success>true</Success></WorkflowPublish>` (XML). `false` means the whiteboard has no `error_free` value.

12.4 find_constraints

`GET api/find_constraints?file=<path>[&wf_id=<workflow id>]`

Analyzes a workflow file present on the Orchestrator host and returns the list expected by `import_with_constraints`, with the possible choices for each conflict:

```
[{"filename": "<path>"}, {"add as revision": null}, {"subwf constraints": {}},
 {"action template constraints": {}}, {"remote node constraints": {}},
 {"missing plugins": null, "Auto-enable missing plugins?": null}]
```

It does not check that the file exists: a wrong path returns the same empty lists.

12.5 import_with_constraints

`POST api/import_with_constraints`, JSON body: an array of six objects, in this order:

```
[{"filename": "/opt/aspera/orchestrator/var/run/orchestrator/workflows/import/my_workflow.yml"},
 {"add as revision": 239},
 {"subwf constraints": {}},
 {"action template constraints": {"Search Files": "Map to the existing Global Template"}},
 {"remote node constraints": {"Baton": "Map to the existing Remote Node"}},
 {"Auto-enable missing plugins?": "true"}]
```

Item	Content
`filename`	Path of the file on the Orchestrator host.
`add as revision`	Id of an existing workflow to add a revision to, or `null` to create a new workflow.
`subwf constraints`	Sub-workflow name to choice: `Map to the Existing Sub-Workflow`, `Create a new Sub-Workflow with a unique name`, `Add a new revision to the existing Sub-Workflow`.
`action template constraints`	Global template name to choice: `Map to the existing Global Template`, `Create a new Global Template with a different name`, `Override the existing Global Template with this new template`.
`remote node constraints`	Remote node name to choice: `Map to the existing Remote Node`, `Create a new Remote Node with a different name`, `Override the Existing Remote Node with this importing node`.
`Auto-enable missing plugins?`	`true` or `false`.

Response: the workflow as JSON. The workflow is not published.

12.6 initiate

`POST api/initiate?format=json`, JSON body `{"workflow_id": <id>, "external_parameters": {"<name>": <value>, ...}}`. Query parameters:

Parameter	Content
`synchronous`	`true`: wait for the end of the work order.
`timeout`	With `synchronous`: maximum wait, in seconds.
`explicit_output_step`, `explicit_output_variable`	With `synchronous`: return only this output.
`explicit_output`	Same, as `<step name>:<output name>`.
`on_error_return_code`, `on_failed_return_code`	HTTP status to return when the work order ends `Error` or `Failed`.
`tags`	Comma-separated tags of the work order.

Give the value of an `array` or `hash` parameter as a string, in JSON: `{"files": "[\"/a.mxf\", \"/b.mxf\"]"}`. A JSON array as value fails with status 500 (`undefined method 'gsub' for an instance of Array`, `MultiType.convert_value_array`), observed on 4.1.6.

12.7 Other workflow endpoints

Endpoint	Content
`GET api/workflows_list`	All workflows: id, name, portable id, published and latest revision.
`GET api/workflow_inputs_spec/<id>`	Runtime parameters: name, type, optional, default value.
`GET api/workflow_outputs_spec/<id>`	Outputs: `<step name>:<output name>`, type.
`GET api/work_orders_list/<workflow id>`	Work orders of a workflow.
`GET api/work_order_status/<id>`	Status of a work order.
`GET api/work_order_output/<id>`	Output values of a work order (XML: one `variable` element per output).
`GET api/work_order_cancel/<id>`	Cancel a work order.

12.8 ascli commands

Operation	Command
Import	`ascli orchestrator workflows import <file>`
Import with constraints, or as a revision	`ascli orchestrator workflows import_with_constraints @json:'{"filename":"<path on server>","add_as_revision":<id>}'`
Export	`ascli orchestrator workflows export <id> --out.file=<file>`
Export with dependencies (`.wkf`)	`ascli orchestrator workflows export <id> @json:'{"dependencies":true}' --to-folder=<folder>`
Publish	`ascli orchestrator workflows publish <id>`

Operation	Command
Inputs, outputs	`ascli orchestrator workflows inputs <id>`, `ascli orchestrator workflows outputs <id>`
Start	`ascli orchestrator workflows start <id> [<parameters>] [<execution>]`, where execution is `{"synchronous":true}` or `{"step":"<step name>","variable":"<output name>"}`
Work orders of a workflow	`ascli orchestrator workflows workorders <id>`
Work order	`ascli orchestrator workorders status <id>`, `ascli orchestrator workorders output <id>`, `ascli orchestrator workorders cancel <id>`
Journal, monitors, queues, resource pools	`ascli orchestrator journal files <book>`, `ascli orchestrator monitors list <type>`, `ascli orchestrator queues list`, `ascli orchestrator pools status <name>`
Query parameters of an API call	`--query=@json:'{...}'`; `--query=help` lists them, from the OpenAPI description

Chapter 13

Known issues

Observed in Orchestrator 4.1.6:

- A negative `Synch_factor` is never met (see Prerequisites and branches).
- When a `Synch_factor` other than `0` has no attributes `x` and `y`, the designer opened from the list of workflows generates `workflow.addFigure(<join>, , );` (`WhiteboardsHelper#draw_operator`): the JavaScript does not parse, and the canvas is empty. The Revisions tab (`draw_xmlconnections`) skips the join symbol and shows the graph.
- `find_constraints` returns empty constraints for a file that does not exist.
- `LocalFileWatcher` keeps a history of past triggers only when `remove_file` is `false`, not when it is null (the database default): a template that leaves it blank detects the same files again at each scan, whatever `trigger_type`.
- `export_workflow` exports dependencies for any non-empty value of `export_with_dependencies`, including `false`.
- `import_workflow` and `import_with_constraints` should refuse a file whose element 4 lists dependencies when element 5 is empty, but the test also requires element 5 to be non-empty: such a file is imported, with template attributes as written.
- `import_workflow` does not import the remote nodes of a `.wkf` package.
- `export_workflow` has no route without the workflow id in the path, although its comment documents `?id=`.
- The `.wkf` package produced by `export_workflow` with dependencies contains an empty copy of itself (the archive is built in the folder it packs).
- API errors return the exception stack trace, and often status 404 or 500 regardless of the cause.
- The example files of the product (`docs/workflow_examples`) contain `<Failed_retry_for_unit>`, while the parser reads `<Failed_retryForUnit>`: that setting is ignored. The same applies to `<Error_retry_for_unit>` (correct: `<Error_retryForUnit>`). Such workflows also exist on servers when imported from a `.wkf` package of unknown origin: the retry period is silently ignored. The designer writes these elements: opened from the list of workflows, it draws the retry settings of the published revision with properties `Failed_retry_for` and `Failed_retry_for_unit` (`WhiteboardsHelper#set_retry_for_branch`), while its retry tab and the parser use `Failed_retryFor` and `Failed_retryForUnit`. So the retry tab shows Retry Duration empty, and a save writes `<Failed_retry_for>` and `<Failed_retry_for_unit>`: the retry period is lost; attempts and delay are kept (observed with the serializer of the designer). The Revisions tab draws from the XML and shows the period.
- The inline validation endpoint `POST external_calls/validate?id=<workflow id>` (`ExternalCallsController#a` `call`, called by `ascp` for each file) is not exempt from the CSRF protection enabled in 4.1.5: the request, without CSRF token, is answered with a redirection to the logon page (HTTP 302), and no work order starts. 4.0.1 had the protection off. The workflow itself can be tested by starting it with runtime parameters `File` (a JSON string) and `File_path`.
- `OrchestratorAlertEntry` does not save its header (`execute` passes only the category and the message).
- At publish, a designer note is looked up by its `note_id` in all workflows (`Note.find_by(note_id:)`): a note whose id is also used in another workflow updates the note of that workflow, and is not shown in this one. The designer generates UUIDs; in a hand-written file, use a UUID too, not `note1`.

End of document